Linux Systems and Performance

PART 2: CPU Performance and Scheduling

Week 4: CPU Architecture and the Linux Scheduler

Youngjae Kim, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr/

E-mail: youkim@sogang.ac.kr



Roadmap of This Week

PART 1

What does the CPU look like?



- Topology, SMT
- big.LITTLE, capacity
- PELT, uclamp

PART 2

Where does data live?



- Our DynamIQ SoC, caches
- TLB, warm vs. cold
- False sharing, NUMA

PART 3

Who runs next, and where?



- Classes, balancing
- Misfit, wake-up, EAS
- cgroups, observing

PART 4

How does webOS stay smooth?



- Three kinds of work
- Policy mapping
- Case: slow app launch



Many Cores
Where should each task run?



Many Tasks
Who gets the CPU next?



Many Deadlines
Frames & audio must be on time

Week 1 Process / FD / Syscall → **Week 2** Interrupt / Signal → **Week 3** Event-driven I/O / IPC → **Week 4** **CPU & Linux Scheduler**

How Does This Relate to webOS?

One TV, many tasks at once

- **Remote key press** → UI must react now
- **4K video** → decode control every frame
- **Audio** → continuous stream, no gaps
- **Web app** → JS, layout, paint
- **Background** → app update, logging, indexing

CPU cores (shared)

Core 0: LSM (compositor)

Core 1: WAM renderer

Core 2: Media pipeline

Core 3: Update service

Runnable, waiting for a CPU:

Audio | SAM* | Logger | Input

Which task runs where, and when?

16.7 ms per frame

60 Hz: 1/60 s ≈ 16.7 ms

Smooth: a new frame every 16 ms



Jank: one frame takes 35 ms



One late frame = the old image stays up for 2+ refreshes → a visible stutter. If the renderer is runnable but not running, this happens **even if the renderer itself is fast** (next slide).

1. **CPU (Part 1)** — Topology, big.LITTLE
2. **Caches (Part 2)** — Cost of placement
3. **Scheduler (Part 3)** — Classes, EAS, cgroups
4. **webOS (Part 4)** — Priorities, jank

** **SAM** = System and Application Manager: the webOS service that launches apps and tracks their foreground / background state.*

Why Does Bad Scheduling Matter? Missing the 16.7 ms Deadline

60 Hz display = 60 refreshes per second

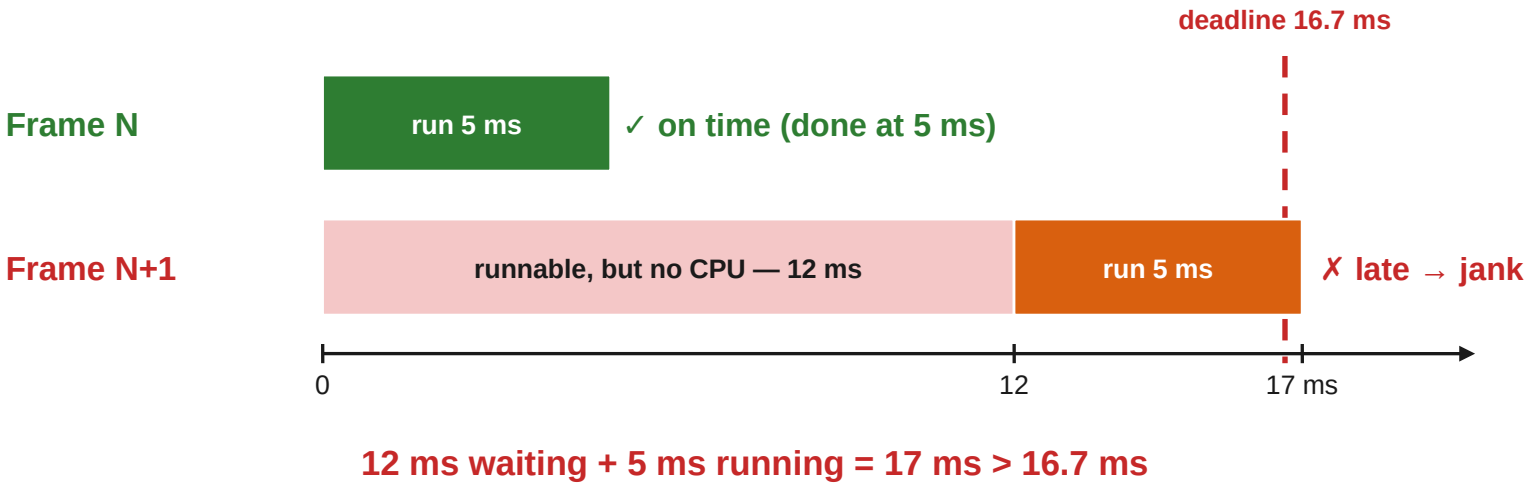
1 / 60 s ≈ 16.7 ms

time budget for one frame

What went wrong in frame N+1?

- **Renderer is not slow:** needs only 5 ms of CPU
- **It waited in the runqueue:** runnable for 12 ms while other tasks held the core
- **Deadline still passes:** done at 17 ms → shown one refresh late → stutter

Renderer needs only 5 ms of CPU per frame



12 ms waiting + 5 ms running = 17 ms > 16.7 ms

Where can the 12 ms come from?

- Other runnable tasks on the same core with equal or higher priority
- Placement on a busy core (or a slow LITTLE core) — Part 3
- Wake-up from deep idle or ramp-up from low frequency

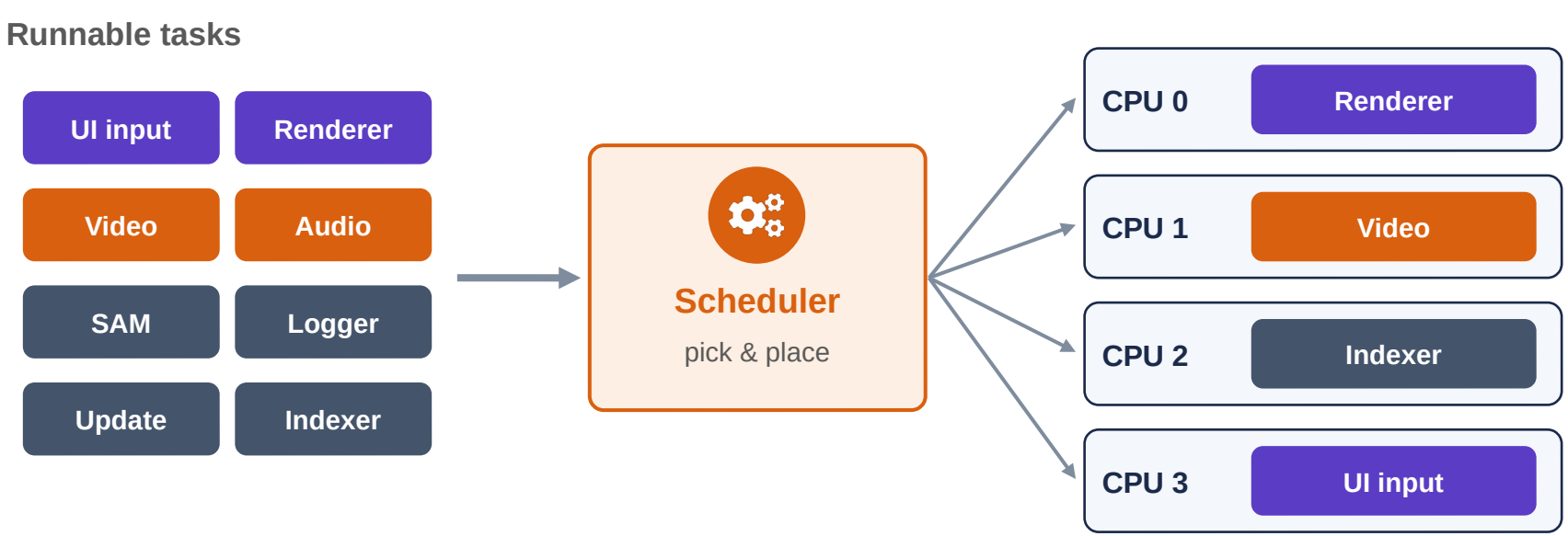
Low CPU utilization or a fast app is not enough — **scheduling latency alone can cause jank.**

PART 2: CPU Performance and Scheduling

- **Week 4**

- **CPU Topology and big.LITTLE**
  - What does the hardware look like to the scheduler?
- **Our SoC: Caches and the Memory Hierarchy**
  - Where does data live, and what does moving a task cost?
- **Linux Process Scheduling**
  - How does Linux organize runnable tasks and place them on CPUs?
- **Scheduling in webOS**
  - How can webOS keep UI and media smooth under load?

The Problem: Many Runnable Tasks, Few CPUs



Three decisions

1. **WHO runs next?**
   order, priority
2. **WHERE does it run?**
   which CPU / core
3. **HOW LONG?**
   time slice, preemption

Requirements

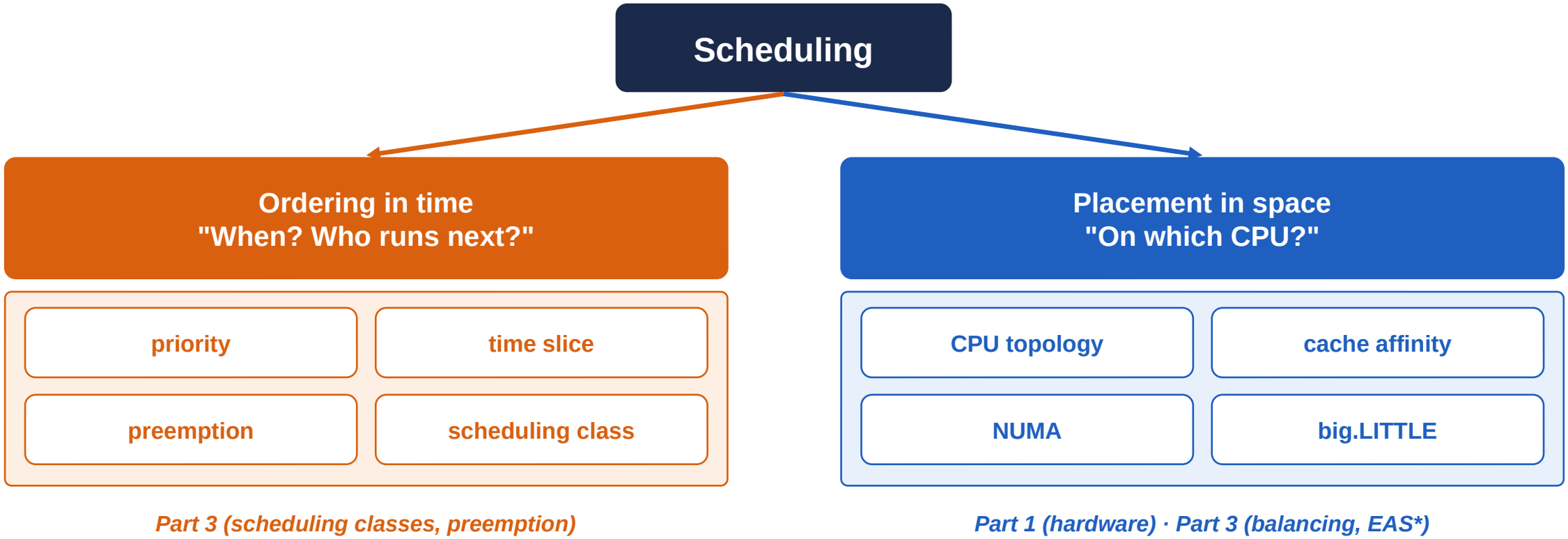
- Responsive UI (every frame on time)
- Media never starves
- Background still makes progress
- Low power on a small SoC

Key idea

Scheduling = ordering in time + placement in space.
The hardware (cores, caches, memory) decides how expensive each placement is (next slide).

Scheduling Has Two Dimensions: Time and Space

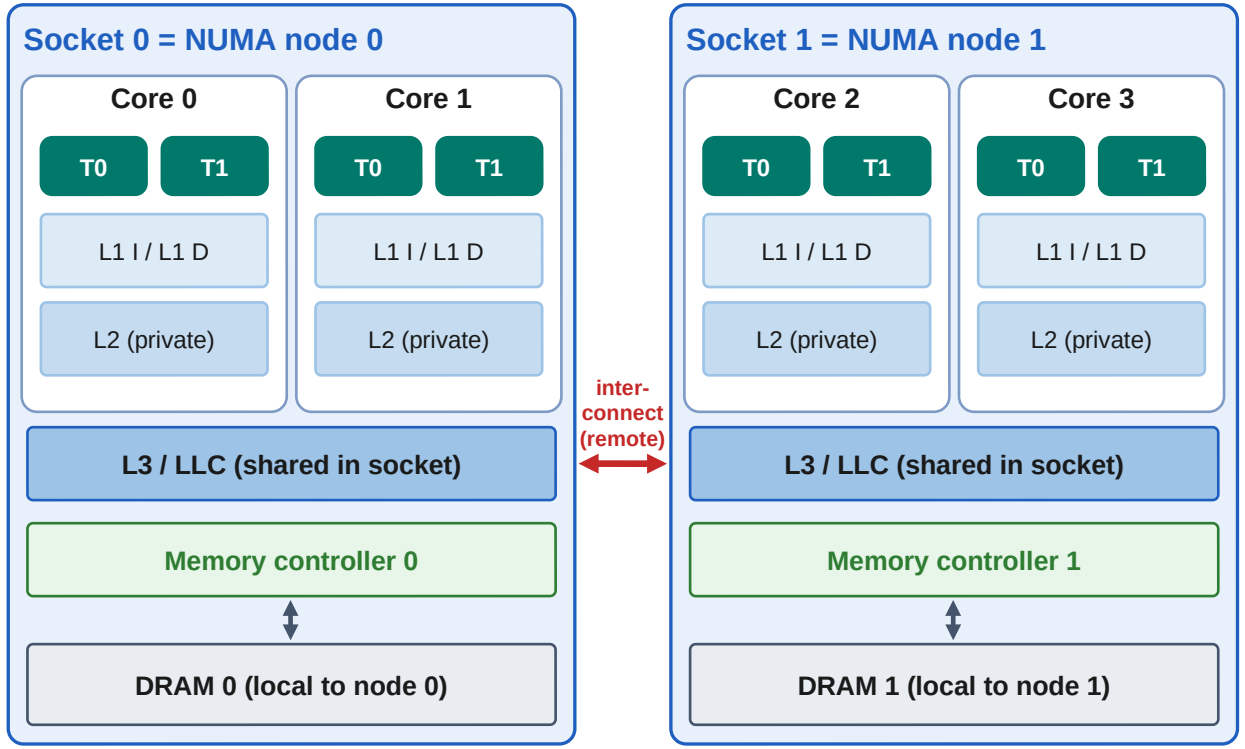
Common view: "Which process runs next?" **On multicore, the scheduler answers two questions at once.**



** EAS = Energy-Aware Scheduling: on big.LITTLE, picks the CPU that meets the task's needs at the lowest energy (Part 3).*

Scheduling = ordering in time + placement in space — and the cost of placement is set by the hardware. That is why Part 1 studies CPU architecture first.

CPU Topology: The General Picture



NUMA (multi-socket server): each node has its own memory controller; DRAM of the other node is remote — slower, via the interconnect. **UMA**: one memory controller, same latency for every core — a single socket, or our TV SoC.

Level	Shares	Linux sysfs (full path, CPU 0)
HW thread (SMT)	pipeline, L1, L2	`/sys/devices/system/cpu/cpu0/topology/thread_siblings_list`
Core	private L1 (+L2 typ.)	`/sys/devices/system/cpu/cpu0/topology/core_id`
Cluster / LLC	L2 or L3	`/sys/devices/system/cpu/cpu0/topology/cluster_cpus_list`
Package	memory controller	`/sys/devices/system/cpu/cpu0/topology/physical_package_id`
NUMA node	local DRAM	`/sys/devices/system/node/node0/`

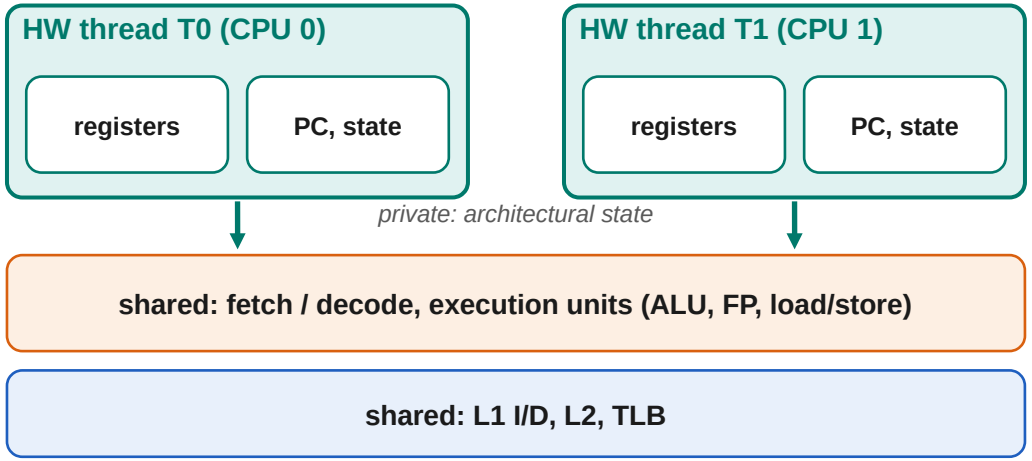
Our target (next slides): Arm big.LITTLE SoC

- No SMT: 1 core = 1 logical CPU
- Two core types: big and LITTLE
- Shared L3, one memory node (UMA)

The scheduler builds its **scheduling domains** from this topology (→ Part 3).

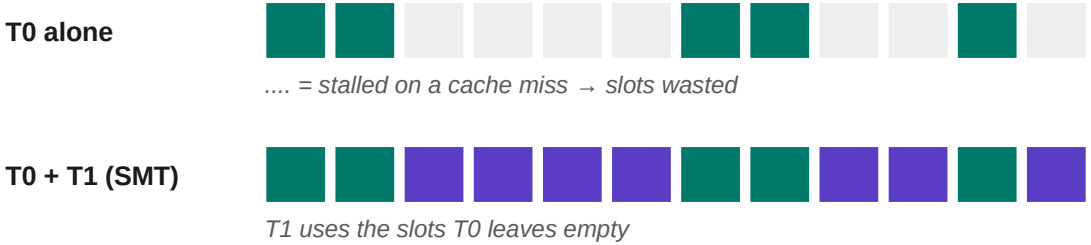
SMT: Two Hardware Threads Share One Core

One physical core with SMT (2 hardware threads)



The OS sees 2 logical CPUs — but only 1 core's resources.

Why it helps: filling idle issue slots



- **Typical gain:** ~10–30% more throughput, not 2× (workload-dependent)
- **Two busy siblings** compete for the same units and caches

Scheduler impact

- Spread to other physical cores first (SMT domain)
- Busy siblings ≠ 2 full cores

Check on Linux

- `lscpu`: "Thread(s) per core: 2"
- `/sys/devices/system/cpu/smt/active`

Our TV SoC

- Cortex-A55 / A76: no SMT
- 1 core = 1 logical CPU

SMT doubles the logical CPUs, **not the hardware** — siblings share one core's pipeline and caches.

Reading the Topology: Linux sysfs

sysfs = virtual filesystem (mounted at `/sys`) where the kernel exposes devices and topology. Per CPU: **`/sys/devices/system/cpu/cpuN/topology/`**

1 thread_siblings_list

```
$ cat cpu0/topology/thread_siblings_list
0,1
```

CPU 0 and CPU 1 are SMT siblings of **one physical core** — not independent cores.

2 core_id

```
cpu0 → 0    cpu1 → 0
cpu2 → 1    cpu3 → 1
```

Core 0 = {CPU 0, 1}, Core 1 = {CPU 2, 3}. **Unique only within a package** — read with physical_package_id.

3 cluster_cpus_list

```
$ cat cpu0/topology/cluster_cpus_list
0-3
```

CPUs 0–3 form one cluster (natural on Arm). **Cluster ≠ cache sharing** in every SoC: also check cpu*/cache/.

4 physical_package_id

```
CPU 0–31  → 0   (socket 0)
CPU 32–63 → 1   (socket 1)
```

Which socket a CPU belongs to — e.g., a 2-socket server.

5 /sys/devices/system/node/

```
$ cat node/node0/cpulist
0-15
# node0 also owns its local DRAM
```

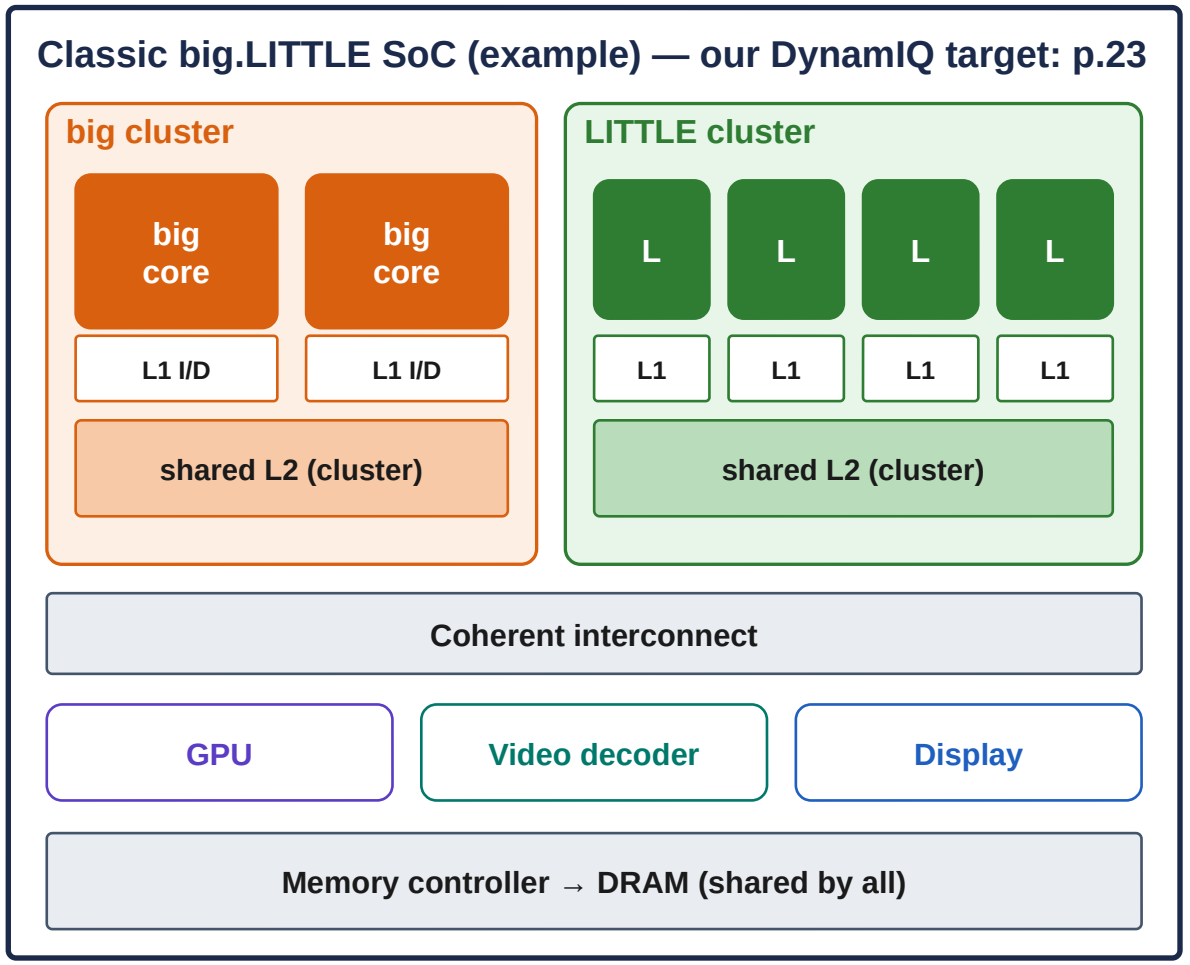
NUMA node = CPUs + their local DRAM. Node 0 → node 1 memory is a **remote access** (placement matters).

On an Arm TV / mobile SoC

- No SMT: **1 core = 1 logical CPU**
- Clusters matter: LITTLE CPU 0–3, big CPU 4–7
- "CPU 2 and CPU 6 are idle" is not enough: **capacity and energy cost differ → EAS**

sysfs shows the scheduler's map of the hardware: who shares a core, a cluster, a socket, a memory node.

Heterogeneous CPUs: Arm big.LITTLE



	big	LITTLE
Performance	high IPC*, high freq	lower IPC*
Power	high	low
Good for	bursty, latency-critical	background, light periodic
cpu_capacity	1024 (max)	e.g., ~300–500

Same ISA, different cores

big and LITTLE run the same Arm instruction set, so the OS can move any task between them — placement becomes the scheduler's job.

Why GPU / video decoder / display?

A TV SoC: pixels are decoded and composed in hardware, but the CPU still runs control, apps, UI and the compositor.

** **IPC** = Instructions Per Cycle (why big is higher: next slide)*

On asymmetric CPUs, "which core" changes both **speed and power** → placement becomes a policy decision (EAS, uclamp).

Why big Has Higher IPC than LITTLE

big (e.g., Cortex-A76)

wide, out-of-order

- **Wide:** decodes ~4 instructions per cycle and issues them to several units (ALU, FP/SIMD, load/store) at once — the "wide exec units" of p.14
- **Out-of-order:** while one instruction waits on a cache miss, independent later ones run first; large buffers (ROB) hold tens to 100+ instructions
- **Big, accurate branch predictor:** fewer pipeline flushes
- **Larger caches, stronger prefetchers:** fewer misses to begin with (p.41)

Waiting time is filled with other work → many instructions finish per cycle.

LITTLE (e.g., Cortex-A55)

narrow, in-order

- **Narrow and in-order:** issues at most ~2 instructions per cycle, strictly in program order
- **One stall stops everything behind it:** on a cache miss the pipeline mostly waits for the data
- **Smaller buffers, predictor and caches:** more misses and mispredictions

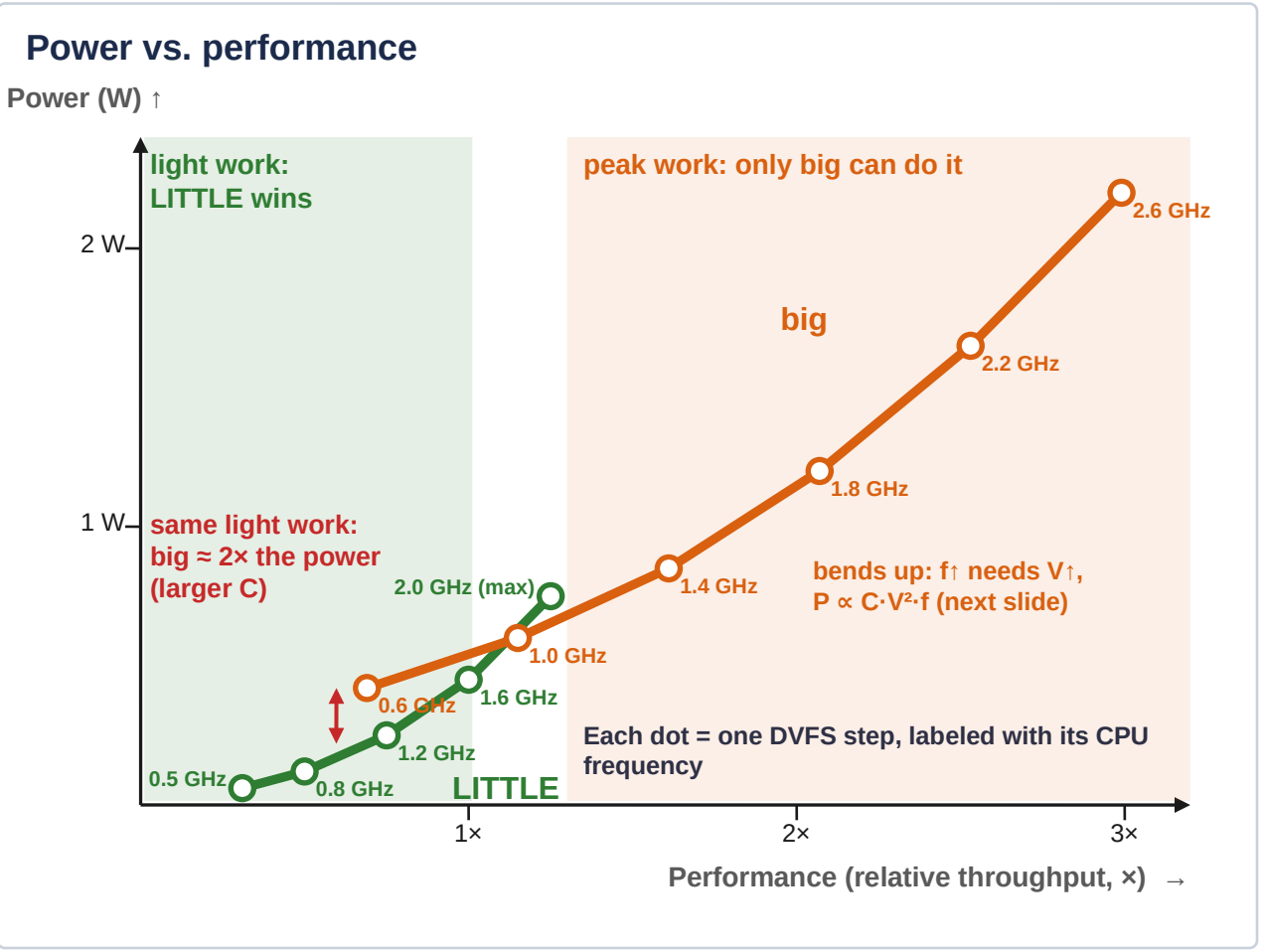
Simpler and smaller → far less energy per instruction, but lower IPC.

Performance ≈ IPC × frequency: big is higher on both (and burns more power); LITTLE is lower on both but much more energy-efficient. The gap grows on memory-bound code — big hides stalls, LITTLE waits.

IPC = Instructions Per Cycle

big = **wide + out-of-order** → high IPC; LITTLE = **narrow + in-order** → low IPC, low energy.

Why big.LITTLE? One Core Cannot Be Efficient Everywhere



1 DVFS alone is not enough

Slowing a big core saves less once voltage reaches its minimum, and its leakage stays. A small core does light work for much less energy.

2 Real workloads are bursty

Most of the time: background services, audio, network, standby.
Short peaks: app launch, UI animation, JS compile.

3 Area and heat

One big core ≈ the silicon area of several LITTLE cores. In a fanless TV, running light work on LITTLE means less heat and fewer thermal slowdowns.

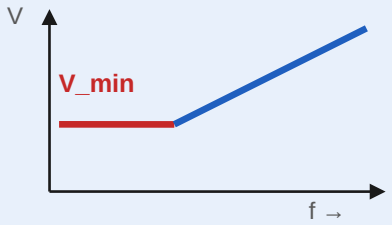
big.LITTLE stitches two efficiency curves together — each piece of work runs where it is cheapest: enough performance at the lowest energy.

Why Not One Big Core with DVFS?

1 DVFS alone is not enough

$$P_{dynamic} \propto C \cdot V^2 \cdot f$$

C: switching capacitance = total capacitance charged / discharged each time transistors switch 0 ↔ 1. **V**: voltage **f**: frequency



below a point, V cannot drop further

→ **p.13:** f↑ needs V↑ → P grows faster than performance (bends up); big core = larger C → higher curve.

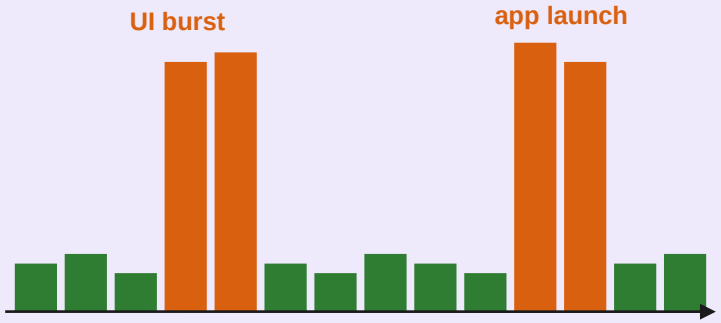
big core

- **OoO engine** — reorders instructions
- **branch predictor** — big history tables
- **large buffers** — ROB, load/store queues
- **wide exec units** — many ALU, FP/SIMD, load/store pipes

still powered at low f → leakage

2 Real workloads are bursty

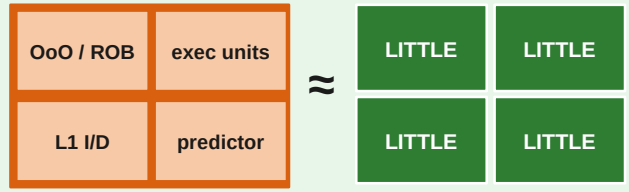
CPU demand over time



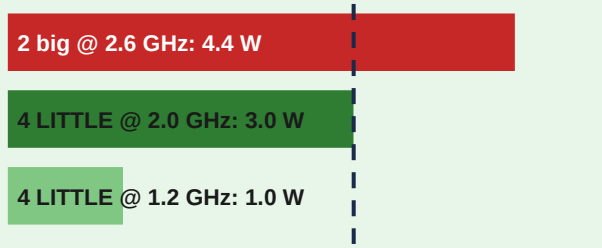
- **Mostly low:** services, audio control, network, standby, logging
- **Short peaks:** app launch, UI animation, JS compile
- **normal → LITTLE, peak → big, then back**

3 Area and heat

Silicon area (roughly to scale)



CPU power vs. a fanless budget (p.13 values)



Task arrives → **How much performance does it need?**

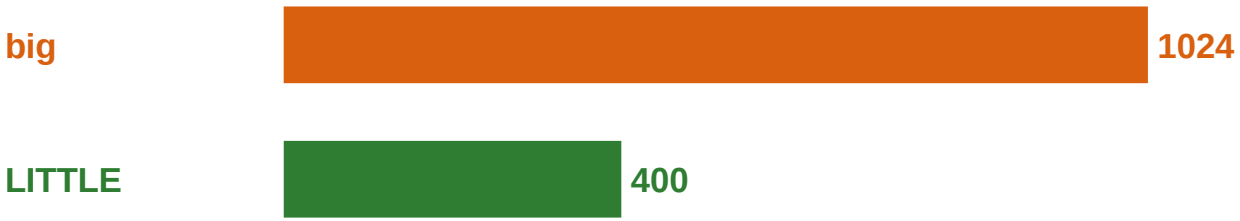
- **light → LITTLE (low power)**
- **heavy / urgent → big**

Hardware offers the two choices; **the scheduler (EAS) picks** — next slides.

"Cheapest" ≠ lowest-power core: **it is the core that meets the needed performance at the lowest energy.**

cpu_capacity: How the Scheduler Sees Core Size

Relative capacity (normalized: largest core = 1024)



400 / 1024 ≈ 0.39 → in the scheduler's view a LITTLE core has about 39% of a big core's capacity (400 is an example; real values differ per SoC).
Why 1024? Not MHz: the scheduler handles utilization and capacity on a fixed-point scale where 1024 = 1.0 (the biggest CPU).

```
$ lscpu                          # overall topology
Architecture:   aarch64
CPU(s):         6
Socket(s):      1
NUMA node(s):   1
$ cd /sys/devices/system/cpu
$ grep . cpu*/cpu_capacity      # scheduler view
cpu0/cpu_capacity:400   ...   cpu3/cpu_capacity:400
cpu4/cpu_capacity:1024
cpu5/cpu_capacity:1024
```

1 Asymmetric system

CPU 0–3 and CPU 4–5 report different capacities → the scheduler must care which core it picks.

2 lscpu vs. cpu_capacity

lscpu: overall topology (CPUs, sockets, NUMA nodes). cpu_capacity: how big each CPU is to the scheduler.

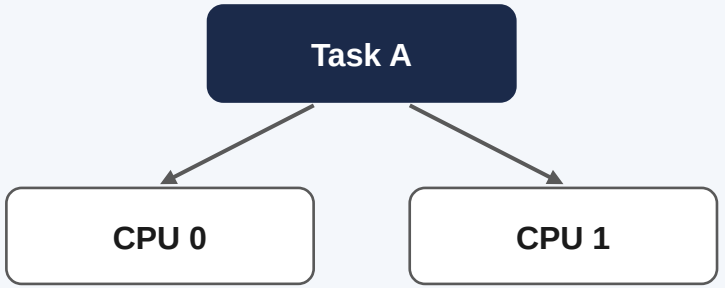
3 Busy task → bigger core, less energy

Util near LITTLE's 400 forces it to max frequency — the steep end of its curve (p.13). A big core at mid frequency can do the same work for similar or less energy. EAS compares both (next slide).

cpu_capacity turns "big" and "LITTLE" into numbers the scheduler can compare with task utilization — and with energy.

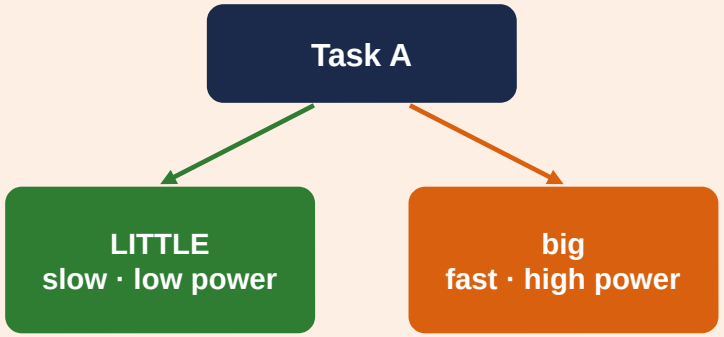
Which Core? Speed vs. Power → EAS and uclamp

Homogeneous CPUs



little difference either way

big.LITTLE



"where" becomes a policy decision

EAS: Energy-Aware Scheduling

- task utilization
- CPU capacity
- energy model

↓

CPU that meets the need at the lowest energy

uclamp (utilization clamping): a floor (min) / ceiling (max) on how "big" the scheduler and DVFS treat a task — details next slide

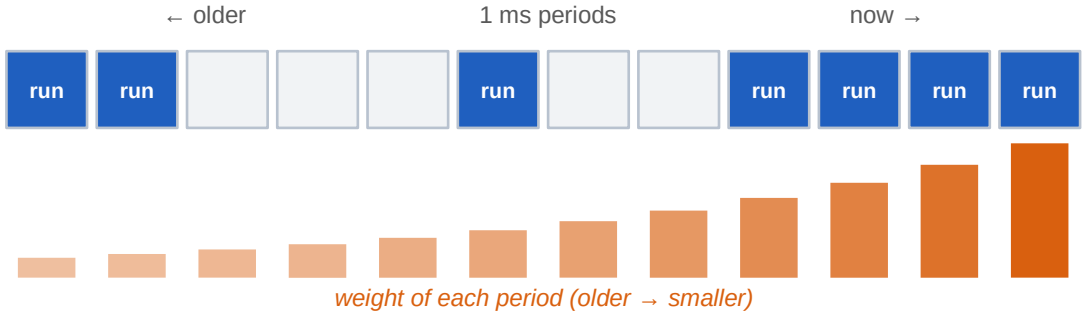
Background logger	low utilization (uclamp.max can cap it)	LITTLE is enough, low frequency
UI renderer	uclamp.min raises its floor	higher capacity / frequency: responds fast

EAS decides where on asymmetric CPUs; uclamp lets user space say how much performance a task needs (details in Part 3).

PELT: How the Scheduler Measures a Task's util

PELT = Per-Entity Load Tracking: the scheduler's running estimate of how much CPU each task (or runqueue / cgroup) used recently — util_avg, 0–1024.

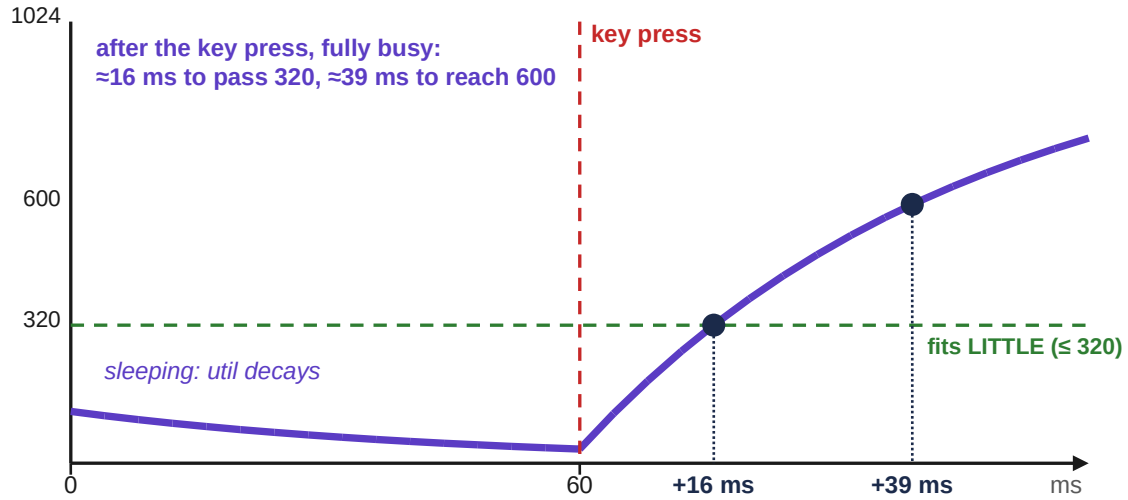
How util is computed: a decaying sum



$$util \approx \Sigma\ (\text{fraction run in period } i) \times y^i\ ,\quad y^{32} = \tfrac{1}{2}$$

Half-life ≈ 32 ms = time for an old period's weight to drop by half:
now 1 · 32 ms ago ½ · 64 ms ago ¼ · 96 ms ago ⅛ … (= $y^{32} = ½$)

Example: renderer util — idle, then a key press



Computed by the Linux kernel (not a HW counter)

- `kernel/sched/pelt.c` computes util from run time — per task, runqueue, cgroup
- Updated at scheduling events: wake-up, sleep, context switch, scheduler tick

Scaled (invariance)

- by capacity and frequency: 1 ms on a slow LITTLE counts less

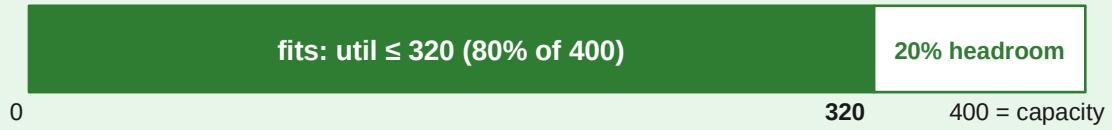
Used by

- EAS / misfit, schedutil, load balancing; see /proc/<pid>/sched

PELT = a decaying history of recent CPU use: **fine for steady load, too slow for sudden bursts** → uclamp (next slide).

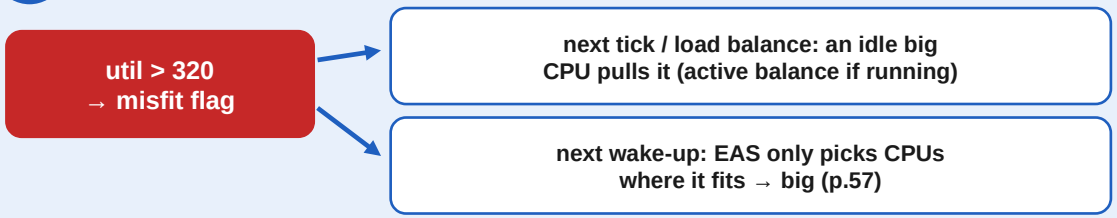
Crossing 320: When Does the Task Actually Move to big?

1 What 320 means



util > ~80% of the CPU's capacity → the task is marked **misfit** on LITTLE (p.56).

2 When it actually moves



misfit = "candidate to move", not an instant move → a few ms more delay is possible.

③ When it does NOT move

- **All big CPUs busy:** no gain, so it stays
- **cpuset limits it to LITTLE (0–3):** big is not allowed (p.61)
- **Low uclamp.max:** it still looks small → never misfit
- **System overutilized:** EAS steps aside, plain load balancing
- **And back:** if util drops on big, EAS may return it to LITTLE at the next wake-up

4 The real problem for bursts



Even an instant move at 320 comes ~16 ms after the key press — the first frame is likely late. **Fix: uclamp.min** treats the renderer as ≥ 512 from the start → big + high frequency at once (p.19–20).

PELT + misfit move a task up **eventually**; for bursts that must make the next frame, use **uclamp.min**.

Why uclamp? Measured Utilization ≠ Desired Policy

EAS decides WHERE from task utilization + CPU capacity + energy model. **But measured utilization (PELT, p.17) is history, not intent.**

UI renderer

`measured util 150`

Mostly idle so far → low utilization. But the user just pressed a key.

Want: run now, on a fast core, at high frequency

→ raise the floor: uclamp.min

Background logger

`measured util 350`

Busy for a while → high utilization. But nobody is waiting for it.

Want: keep it cheap: small core, low frequency

→ cap the ceiling: uclamp.max

Who sets uclamp, and how?

Admin, per group	`echo 50 > /sys/fs/cgroup/ui/cpu.uclamp.min # floor 50%` `echo 20 > /sys/fs/cgroup/background/cpu.uclamp.max # cap 20%`
Admin, per task	`uclampset -m 512 -p <pid>  (util-linux)`
The program itself	`sched_setattr() + SCHED_FLAG_UTIL_CLAMP_*`
System-wide limit	`/proc/sys/kernel/sched_util_clamp_{min,max}`

	Meaning	Typical use
uclamp.min	treat the task as needing at least this much performance (floor)	UI, interactive tasks
uclamp.max	never treat the task as needing more than this (ceiling)	logger, background jobs

Rule: `effective util = clamp(util, min, max)`. Setting another task's clamps needs CAP_SYS_NICE (root); kernel needs CONFIG_UCLAMP_TASK(_GROUP).

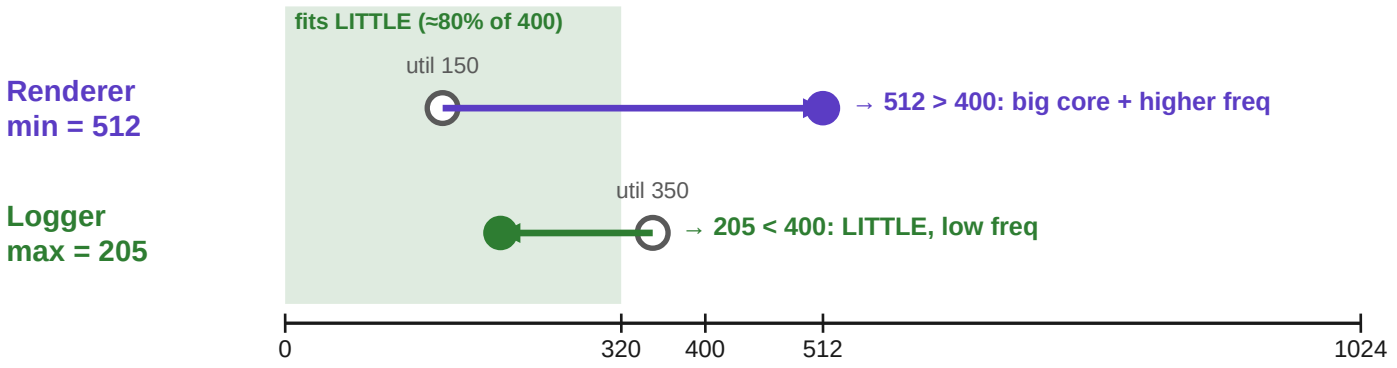
EAS chooses where from what a task did; uclamp lets policy say what the task needs now.

uclamp in Detail: Floors and Ceilings on Utilization

uclamp = utilization clamping. The scheduler tracks each task's utilization (util, 0–1024, via PELT). uclamp clamps it before the scheduler and cpufreq use it:

```
effective util = clamp(util, uclamp.min, uclamp.max)
```

Same 0–1024 scale as cpu_capacity



Is it the "required CPU capacity"?

- **Close:** same 0–1024 units as cpu_capacity
- **`uclamp.min`** = floor: "treat me as needing at least this much" → bigger core, higher frequency
- **`uclamp.max`** = ceiling: "never treat me as bigger than this" → small core, lower frequency
- **A hint, not a reservation:** min = 512 does not reserve a CPU or CPU time (that is SCHED_DEADLINE / RT)

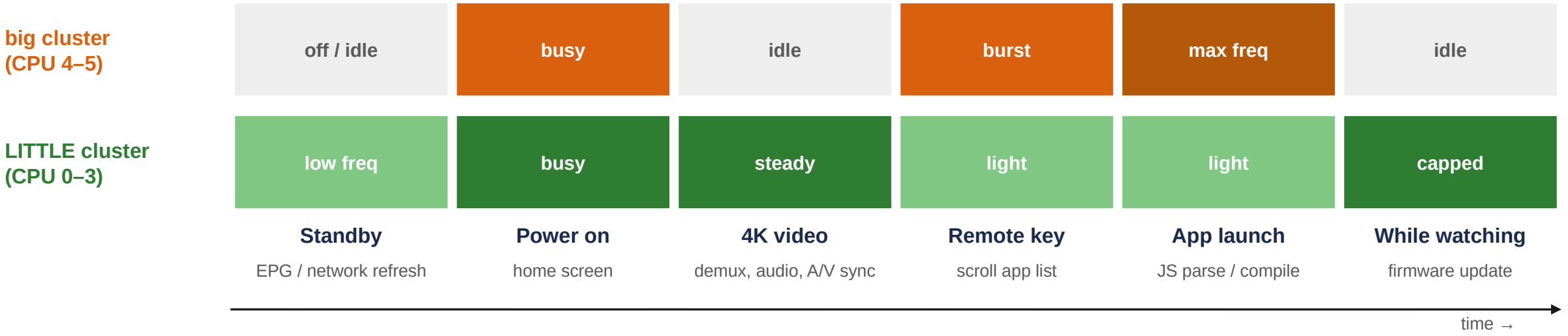
```
// per task (0..1024)
attr.sched_flags = SCHED_FLAG_UTIL_CLAMP_MIN;
attr.sched_util_min = 512;  sched_setattr(0, &attr, 0);
```

```
# echo 50 > /sys/fs/cgroup/ui/cpu.uclamp.min    # percent: 50% ≈ 512
# echo 20 > /sys/fs/cgroup/bg/cpu.uclamp.max    # 20% ≈ 205
$ cat /proc/sys/kernel/sched_util_clamp_max    # system-wide limit
```

cgroup files take a percentage, not 0–1024: **`ui`** 50% → 1024×0.5 = 512, so the renderer (util 150) is treated as 512; **`bg`** 20% → 1024×0.2 ≈ 205, so the logger (util 350) is capped at 205.

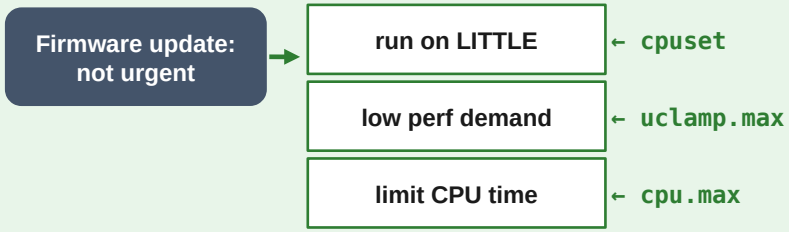
uclamp.min / max set a floor / ceiling on how big the scheduler thinks a task is — **it steers core choice and frequency, but guarantees nothing.**

big.LITTLE in Action: An Evening with a Smart TV (Illustrative)



Kind of work	Where	Mechanism (this week)
Short interactive bursts (UI, input, app launch)	**big, high freq**	EAS sees high util; uclamp.min boosts early
Steady light / periodic (audio, demux, network)	**LITTLE**	EAS: task "fits" the LITTLE capacity
Background bulk (update, indexing)	**LITTLE, capped**	cgroup cpuset + cpu.max / uclamp.max
Pixel-heavy media	**HW blocks**	video decoder / GPU, not the CPU

"LITTLE, capped" = firmware update is not "big → finish fast"



→ progresses slowly — that is fine; **UI and audio come first.**

Most of the evening runs on LITTLE; big cores wake up for the moments the user is waiting.

PART 2: CPU Performance and Scheduling

- **Week 4**

- CPU Topology and big.LITTLE
  - What does the hardware look like to the scheduler?
- **Our SoC: Caches and the Memory Hierarchy**
  - Where does data live, and what does moving a task cost?
- Linux Process Scheduling
  - How does Linux organize runnable tasks and place them on CPUs?
- Scheduling in webOS
  - How can webOS keep UI and media smooth under load?

Our Target: Topology of an Arm big.LITTLE (DynamIQ) SoC

Example SoC: 2 big + 4 LITTLE (cache sizes: one example config)

DynamIQ cluster

CPU 4	CPU 5	CPU 0	CPU 1	CPU 2	CPU 3
big	big	LITTLE	LITTLE	LITTLE	LITTLE
L1 64+64 KB	L1 64+64 KB	L1 32+32 KB	L1 32+32 KB	L1 32+32 KB	L1 32+32 KB
L2 512 KB	L2 512 KB	L2 128 KB	L2 128 KB	L2 128 KB	L2 128 KB

DSU (DynamIQ Shared Unit) — the cluster's shared block

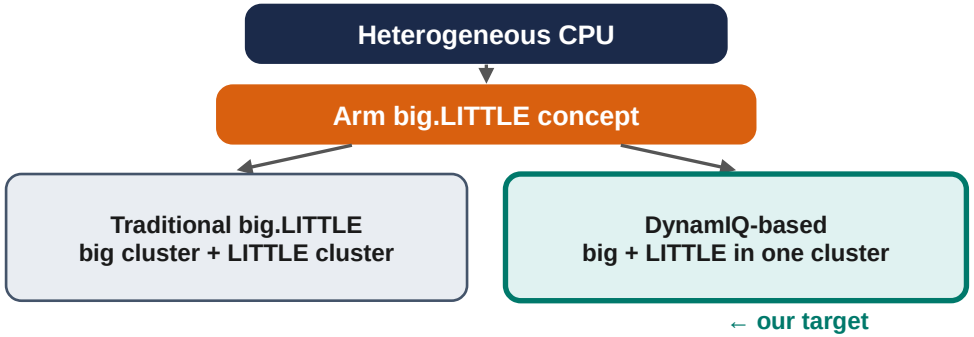
- **L3 cache 2 MB** — shared by big + LITTLE
- **Snoop filter** — snoops only the cores that may hold the line — no broadcast to every core
- **Link to SoC** — coherent interconnect
- **Power control** — cores / cluster on-off

Coherent interconnect

GPU | Video decoder | Display

One DRAM (LPDDR) — a single memory node (UMA)

How the terms relate



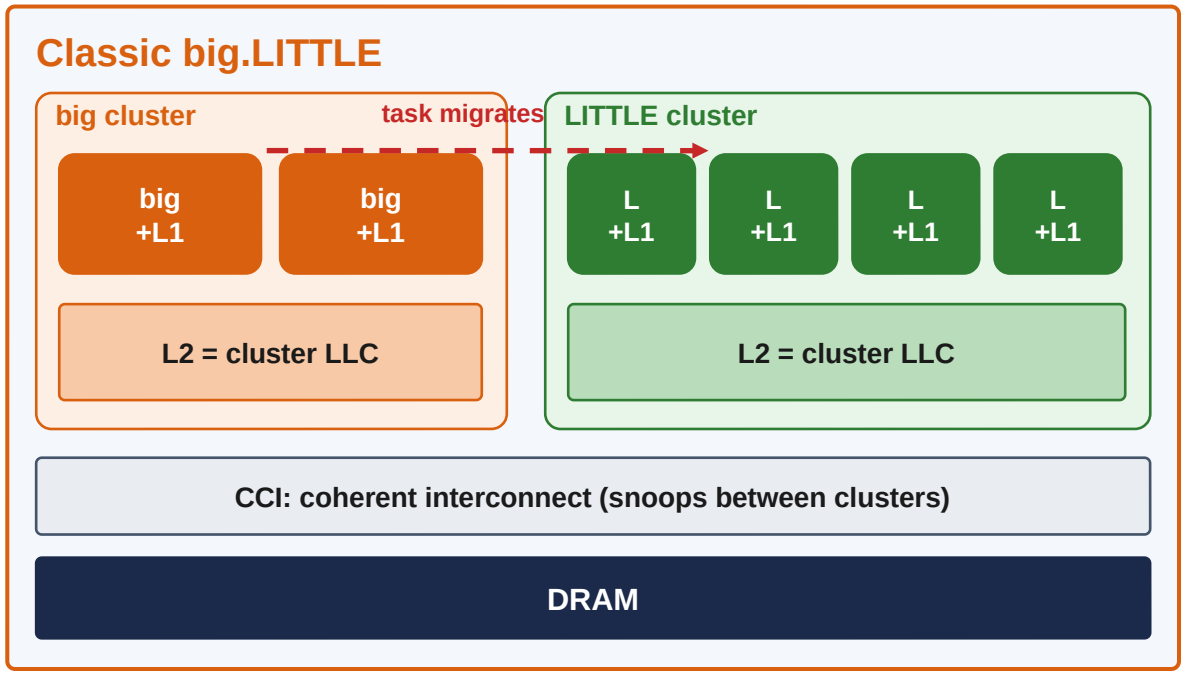
DynamIQ is not a new core type: a cluster architecture that combines different Arm cores.

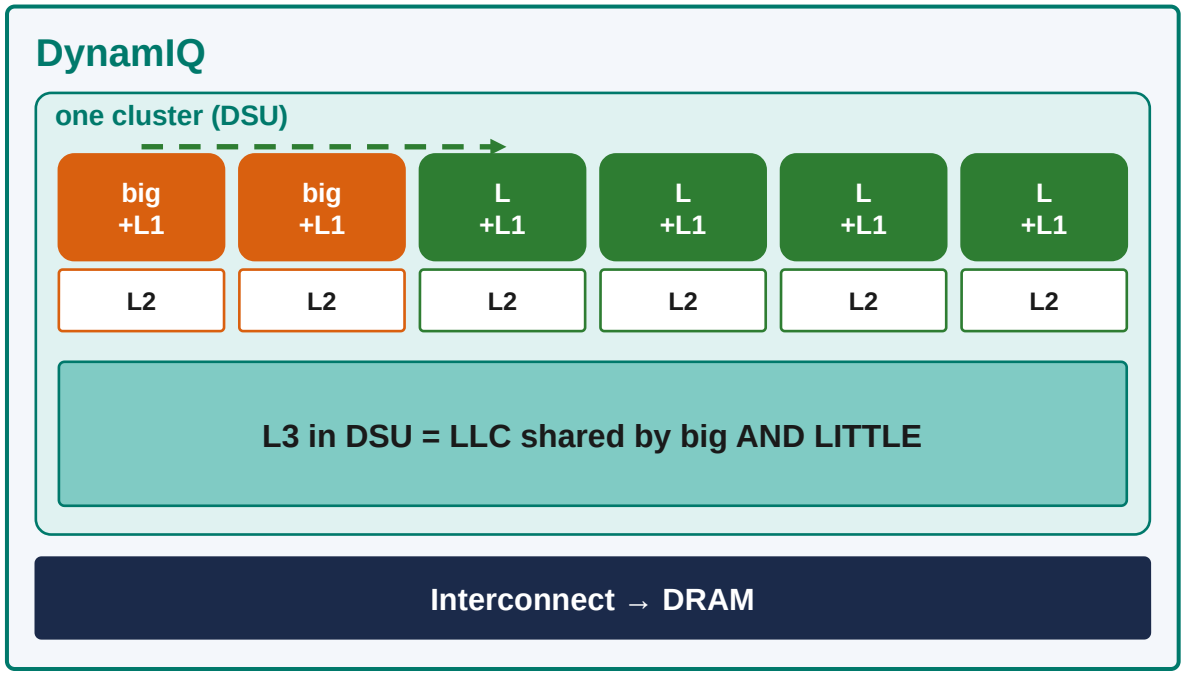
What Linux sees

- `cpu_capacity`: 1024 for big, smaller for LITTLE
- `SD_ASYM_CPUCAPACITY`: kernel flag on the scheduling domain = "CPUs here differ in capacity" → turns on capacity-aware placement, misfit and EAS
- No SMT and no NUMA levels in the domain tree

On our SoC, think: **cores of two types, one shared L3, one memory** — placement is about core type and cache warmth, not NUMA nodes.

big.LITTLE vs. DynamIQ: The Shared-LLC View



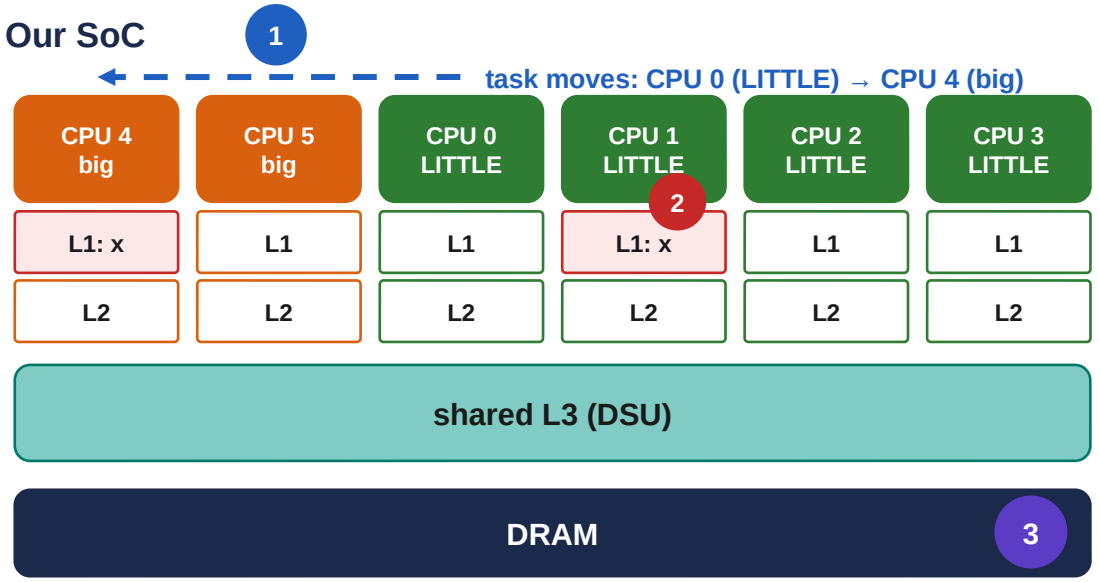


After migration: data stays in the old cluster's L2 → refill from DRAM / cross-cluster snoop

After migration: data is still in the shared L3 → L3 hit instead of DRAM

	big.LITTLE	DynamIQ
Core types per cluster	one type per cluster	mixed in one cluster (Armv8.2+)
Last shared cache for cross-type moves	none on-cluster (DRAM / system cache)	L3 in the DSU
DVFS granularity	per cluster	finer (per core or core group)

From Topology to Caches: Why Open the Box?



1 task moves LITTLE CPU 0 → big CPU 4
2 x in CPU 4 and CPU 1 L1 **3** where the data sits

Every placement decision moves, copies or loses cached data.

From here on we use the DynamIQ layout: private L1 + L2 per core, one shared L3.

1 **A task moves LITTLE CPU 0 → big CPU 4. What does it leave behind?**

→ warm vs. cold caches, migration cost

2 **big CPU 4 and LITTLE CPU 1 both cache x. Who has the latest value?**

→ one 64 B line bounces between cores — false sharing (this part)

3 **Why does it matter so much where data sits?**

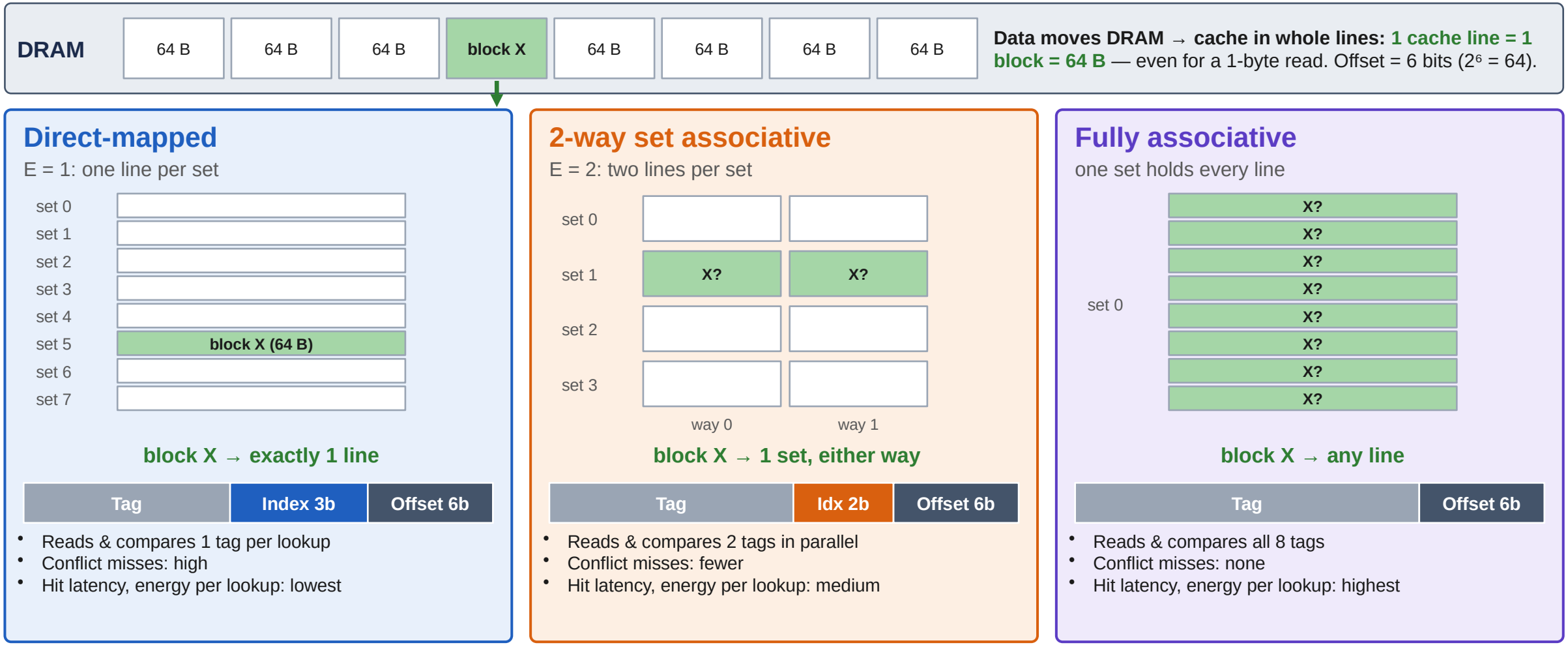
→ L1 ≈ 1 ns vs. DRAM ≈ 100 ns; how caches hold lines (sets, ways, policies)

Next in Part 2

How caches work → Which address → Warm vs. cold → False sharing

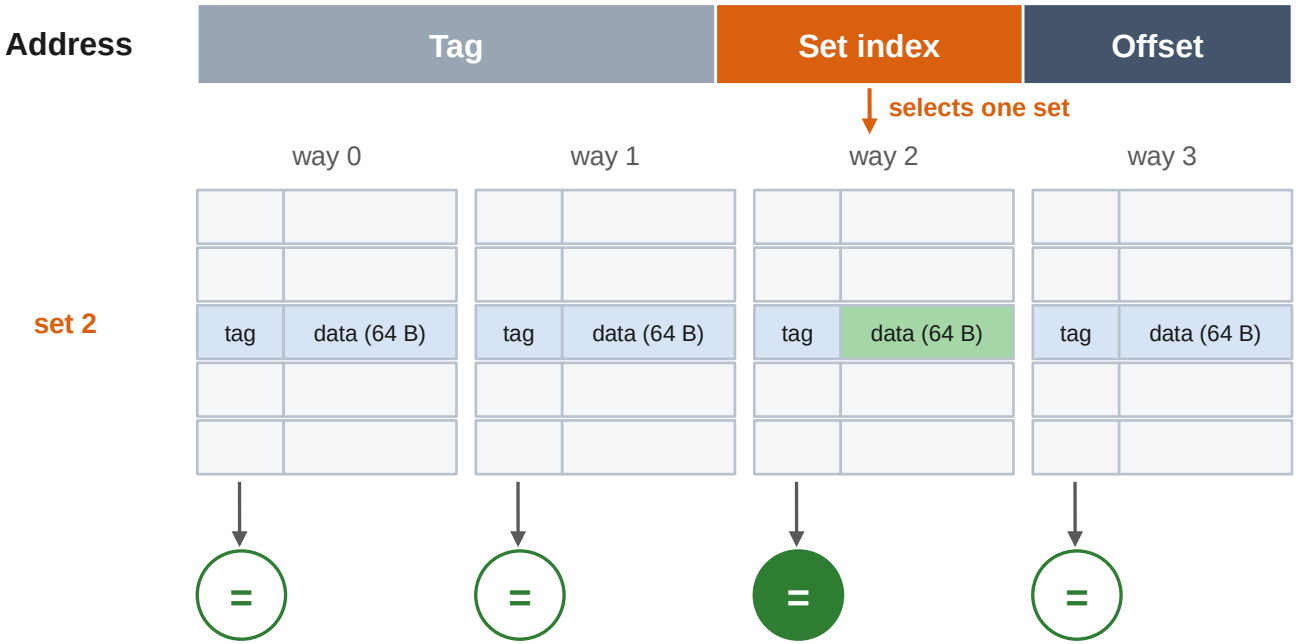
Placement decisions are cache decisions: to judge where a task should run, we must know how caches hold, lose and share its data.

Cache Organization: Where Can a Block Go?



More ways = more tags read and compared on **every** access → slower hits and more energy per lookup, but **fewer conflict misses**. Real CPUs pick the middle: set-associative.

Set-Associative Lookup: How the Hardware Finds a Line



address tag → all 4 comparators in parallel

Hit? (any match)

Way select → offset → bytes

Miss → pick a victim way (LRU / pseudo-LRU) and fetch from next level

Sizing: C = S × E × B

- **S** = number of sets **E** = ways per set (here 4)
- **B** = line (block) size in bytes, e.g. 64 **C** = total size
- `offset bits = log₂ B,  index bits = log₂ S`
- tag = remaining address bits

CHECKPOINT

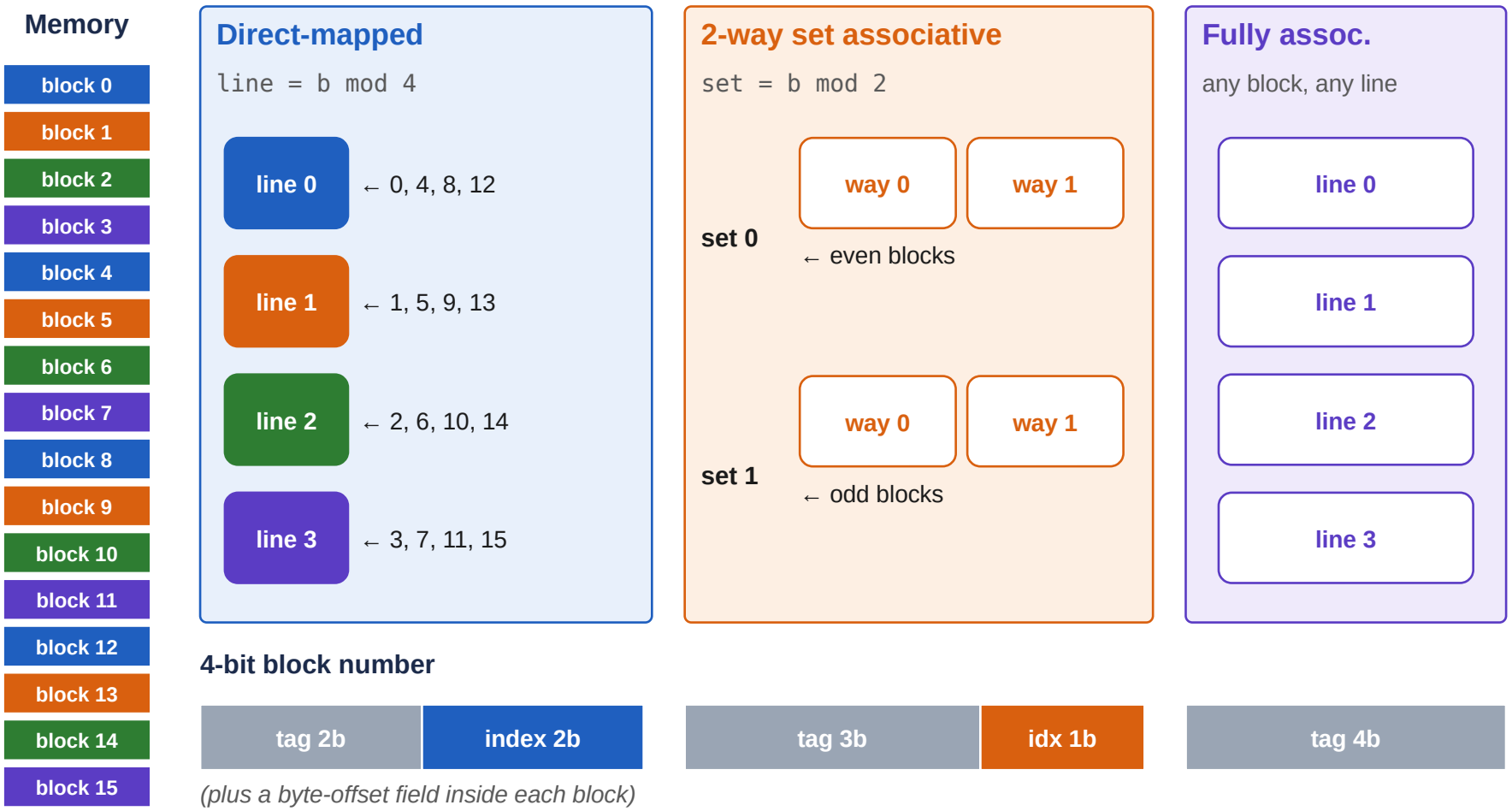
L1D: 32 KB, 8-way, 64 B lines. How many sets? How many index + offset bits?

Answer

- S = 32 K / (8 × 64) = 64 sets
- 6 index + 6 offset = 12 bits
- 12 bits fit inside a **4 KB page offset** → L1 can start the lookup before the TLB finishes (p.42)

Typical: L1 4–12-way, L2 8–16-way, LLC 12–16-way; small TLBs are often fully associative.

Mapping Example: 16 Memory Blocks into a 4-Line Cache



Access	DM	2-way	FA
0	miss	miss	miss
4	miss (evicts 0)	miss	miss
0	miss (evicts 4)	hit	hit
4	miss (evicts 0)	hit	hit
Misses	4	**2**	**2**

Blocks 0 and 4 both map to DM line 0 → conflict misses.

Replacement inside a set: LRU = Least Recently Used.

More placement freedom → fewer conflict misses, but more tags to compare on every access.

Which Organization Do Real Caches Use?

Direct-mapped

1 way · fastest, cheapest — but many conflict misses

- Almost never in modern CPU caches
- Some tiny MCU caches, DRAM caches

Set-associative (N-way)

the middle · good hit rate at reasonable cost

- L1 I/D: 4-way (A55, A76)
- L2: 4-way (A55), 8-way (A76)
- L3 in the DSU: 16-way

used for real caches

Fully associative

every line · no conflict misses — but compares all tags

- Only small structures (tens of entries)
- First-level TLBs (micro-TLBs), small buffers

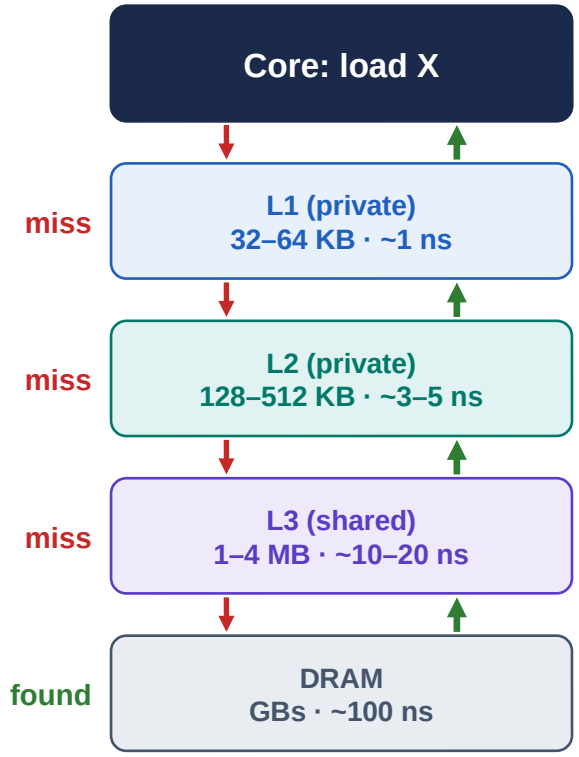
On our SoC (typical configuration; check the core TRM)

L1 I / D	4-way	L2 (private)	4–8-way	L3 (DSU, shared)	16-way	1st-level TLB	fully assoc.
small, must be fast → few ways		bigger, can be a bit slower		largest → more ways cut conflicts		tens of entries; a bigger L2 TLB behind it	

Rule of thumb: the farther from the core, the more ways — L1 stays narrow to keep hits fast.

Real CPUs use **set-associative** caches at every level; direct-mapped is rare, fully associative only for tiny structures like TLBs.

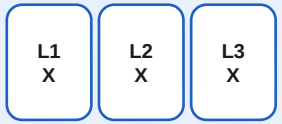
Multi-Level Caches: Where Does a Line Go on a Miss?



A hit at L2 or L3 stops the trip there: the line is copied up into L1.

Which levels keep a copy of X? (inclusion policy)

Inclusive



Every line in L1/L2 is also in L3. + L3 knows everything cached above → cheap snooping. − Duplicate copies use capacity; evicting X from L3 must also remove it from L1/L2.

Exclusive



A line lives in only one level. + No duplicates: capacity ≈ L1 + L2 + L3. − Each L1/L2 eviction moves the line down (lower level = victim cache).

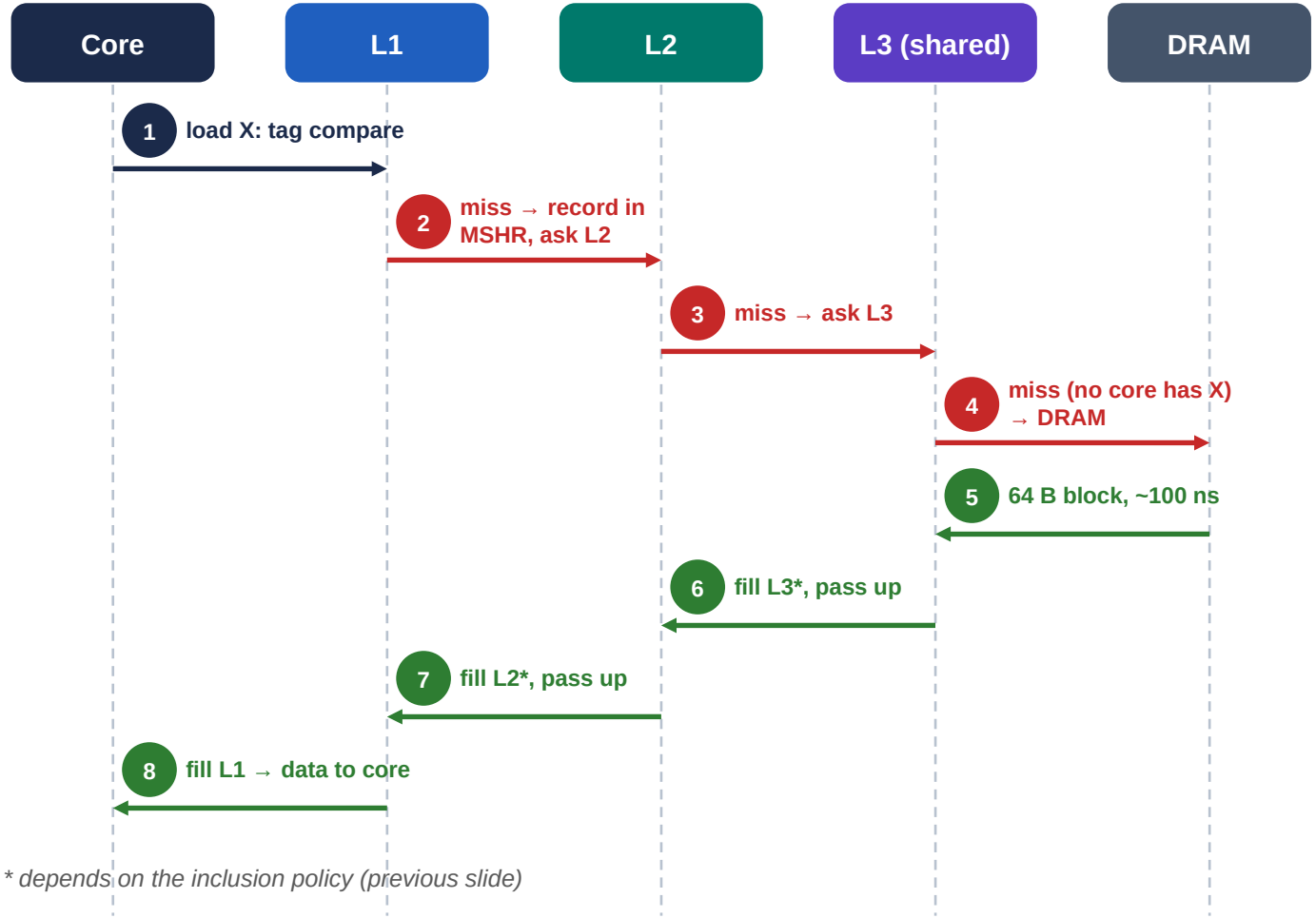
Writes propagate down lazily (write-back)



A dirty line moves one level down each time it is evicted.

A miss walks down the hierarchy until the line is found; the 64 B line then climbs back up, and the inclusion policy decides which levels keep a copy.

Filling a Cache Line, Step by Step



** depends on the inclusion policy (previous slide)*

Making room at each level

- Pick a victim in X's set (LRU / pseudo-LRU)
- Victim dirty? Write it back one level down first
- Then install X: tag + 64 B data, state valid

What is an MSHR?

- **Miss Status Holding Register:** a small table in each cache that remembers every miss still waiting for data (which line, who asked) until the line arrives.

After the fill

- Neighbors in the same 64 B line → L1 hits
- Prefetcher may fetch the next lines already
- Hit in L2: steps 3–6 skipped; hit in L3: 4–5 skipped

One miss = a round trip down the hierarchy and back, evicting a victim at each level it fills — that is why the first touch is slow and the next 63 bytes are cheap.

Why Do Misses Happen? The 4 Cs

1 Compulsory (cold): first touch

cache (empty)



load A → miss

Fix: prefetching, larger lines

2 Capacity: working set > cache

loop over A B C D E F, cache holds 4 (fully assoc., LRU)

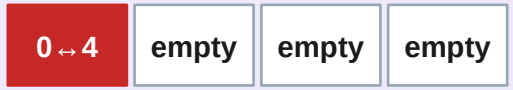


E, F evict A, B — then A, B evict C, D … every access misses

Fix: smaller working set, blocking / tiling, bigger cache

3 Conflict: same set, empty space elsewhere

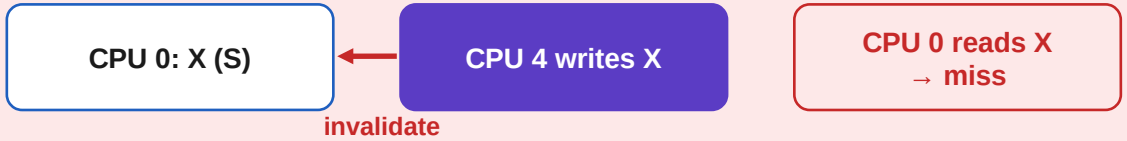
direct-mapped, blocks 0 and 4 → line 0



access 0, 4, 0, 4 … → all misses while 3 lines stay empty

Fix: more associativity, padding / different strides

4 Coherence: another core invalidated it



The line was cached and fits — the miss is caused only by the other core's write.

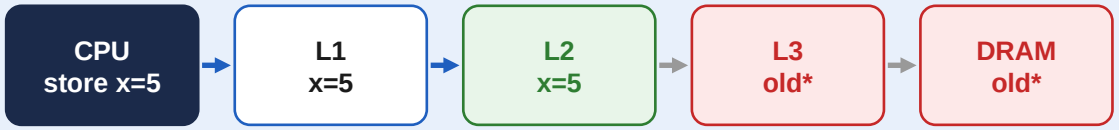
Fix: share less, avoid false sharing (slide 46)

Name the miss before fixing it — the **4 Cs** (Compulsory, Capacity, Conflict, Coherence) each have a different cure.

What About Writes? Write-Through vs. Write-Back

So far: reads, fills and misses. **Now the core writes to a cached line — when does the next level (L2, L3, DRAM) get the new value?**

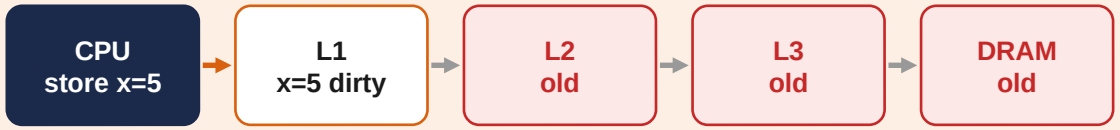
Write-through at L1, write-back below



*L1 → L2 immediately (via write buffer); * L2 and L3 are write-back*

- Every store also goes to L2 right away
- L2 holds the latest value; L1 lines are never dirty
- More L1 → L2 traffic; DRAM still updated lazily

Write-back at every level



dashed = only when the dirty line is evicted, one level at a time

- Store updates L1 only and marks the line dirty
- Evicted from L1 → written to L2; later L2 → L3 → DRAM
- Many stores to one line → one write per level

	Write-through	Write-back
Traffic to next level	every store	only dirty evictions
Next level up to date?	yes — L2 has the new value	no — only L1 holds the newest copy
Other cores' caches up to date?	**no — still needs coherence**	**no — needs coherence**
In a multi-level hierarchy	chosen per level: only L1 → L2 in some designs; L2 / L3 stay write-back	most modern Arm / x86 cores: write-back at every level

Write-back saves bandwidth but makes a private cache the owner of the newest data — the root of the coherence problem.

On a Write Miss: Write-Allocate vs. No-Write-Allocate

Write miss: the core runs `store X = 10`, but the 64 B line holding X is not in L1. Two policies:

Write-allocate — bring the line in first, then write it in the cache

1. store X → miss
2. fetch the whole 64 B line (RFO)
3. install line, write X in the cache
4. mark the line dirty

One 64 B line:



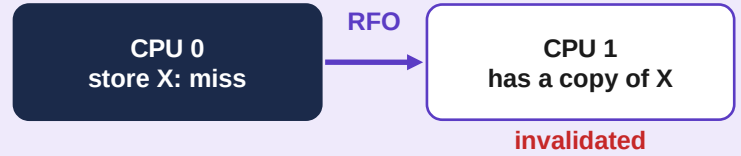
X is only 4 B, but the whole 64 B line is fetched

L1: X = 10 (dirty) | **DRAM: X = old** → **written back only when the line is evicted**

Usually paired with write-back: the common combination in CPUs.

RFO = Read For Ownership

"Give me the line and the right to modify it."



- **Data + write ownership in one request;** other copies invalidated → CPU 0 owns the line (Modified)
- **Where does the 64 B come from?** Same path as a read miss (p.31) — nearest up-to-date copy first: own L2 → L3 or another core's cache (via the DSU) → DRAM last

No-write-allocate — do not bring the line in; send the write down

store X → miss → write goes to L2 / memory → cache unchanged → next load of X still misses

Why would anyone want this? → next slide

Write-allocate + write-back is the common pair: **fetch the line, write it in the cache, write it back later.**

No-Write-Allocate: Why Skip the Fetch?

No-write-allocate

store X → miss → do NOT bring X's line in → send the write to the next level → cache unchanged

later: load X → still a miss

Why? Example: fill a 1 GB array once, never read it again

```
for (i = 0; i < N; i++) A[i] = value;  // 1 GB, written once
```

On a TV: frame buffers and big memory copies are exactly this pattern. Their cache pollution evicts the UI's working set — like background work hurting the UI (Part 3).

With write-allocate: for every 64 B line

read old line (RFO) → allocate → modify → never used → evicted

- **Needless read traffic:** 1 GB is read from DRAM only to be overwritten
- **Cache pollution:** A's dead lines evict data that matters (UI, renderer working set)

With no-write-allocate / streaming stores

- Writes stream straight to memory (via write-combining buffers)
- No read of the old data; cache keeps the useful working set
- In practice: non-temporal stores (Arm `STNP`, x86 `MOVNT`), Arm `DC ZVA` to zero a line without reading it
- libc memset / memcpy often switch to these for large buffers

For data written once and not read soon, skipping the fetch saves DRAM bandwidth and keeps the cache for data that matters.

Non-Temporal Stores on Arm: A Hint, Not a Bypass

In the ISA (AArch64)

```
STP   x0, x1, [x2]    // normal pair store
STNP  x0, x1, [x2]    // non-temporal pair store
// x0–x30: AArch64's 31 general-purpose 64-bit registers
//   (very fast storage inside the CPU)
// x0, x1 = two 8 B values; [x2] = the address held in x2
//   → writes x0 at x2, x1 at x2+8: 16 B in total
// C: __builtin_nontemporal_store(v, p); x86: MOVNTI / MOVNTDQ
```

Who decides what?

Application	big buffer, written once
builtin / compiler	emits STNP
ISA: STNP	says: "not reused soon" (a hint)
microarchitecture	decides the real cache behavior

Normal store (STP) miss

L1 miss → RFO: fetch line → line into L1 → modify → dirty → write-back later

= the write-allocate path (p.34)

Non-temporal (STNP)

STNP → merge stores into a full line → may skip normal L1/L2 allocation → memory system → DRAM

≈ the no-write-allocate path (p.35) — implementation-defined: depends on the core and the memory type

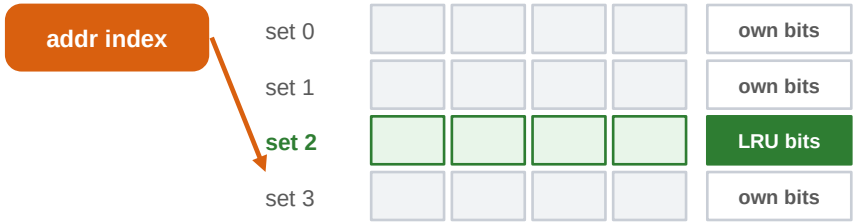
Careful: "STNP = cache bypass" is too strong. STNP only states low temporal locality; which level (if any) allocates the line is up to the implementation.



The ISA gives the hint; **the microarchitecture decides** — measure on your target core before relying on it.

Replacement Policies: LRU and Pseudo-LRU

LRU works inside ONE set



A new line can only go into its own set → the victim is one of that set's 4 ways; every set keeps its own LRU order.

Inside set 2: access — MRU → LRU (order, not position)

A	A				miss
B	B	A			miss
C	C	B	A		miss
D	D	C	B	A	miss
A	A	D	C	B	hit
E	E	A	D	C	miss: evict B

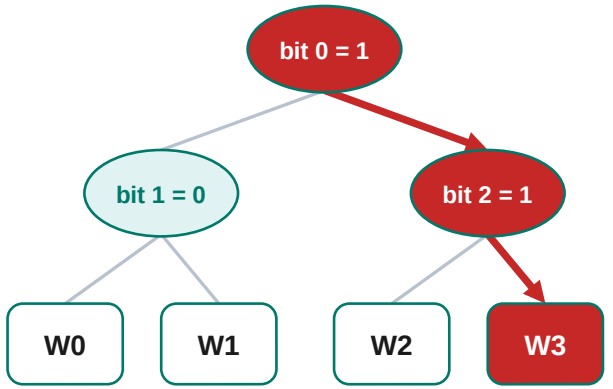
Data never moves between ways; only the set's order bits change. Remembering this full order costs ≥5 bits per set (p.38).

How hardware does it cheaply: tree pseudo-LRU (3 bits per set)

bit 0: {W0,W1} vs. {W2,W3} **bit 1**: W0 vs. W1 **bit 2**: W2 vs. W3

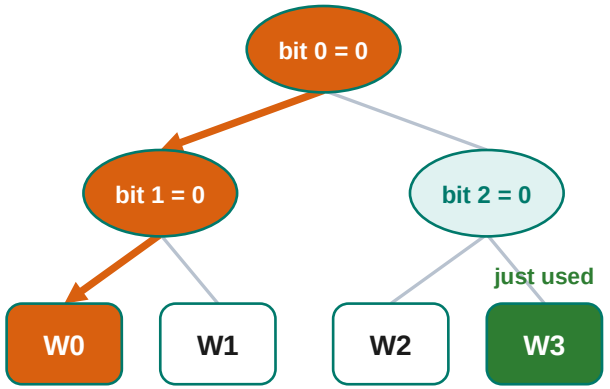
Each bit points to the side used less recently (0 = left, 1 = right). In general E ways need E − 1 bits.

① Pick a victim



bit 0 = 1 → right, bit 2 = 1 → right → victim = way 3

② After filling way 3



bits on W3's path flip AWAY (bit 0 → 0, bit 2 → 0) → next victim: left, bit 1 = 0 → W0

Why not exact LRU? The order must be updated on every access, in one cycle — for 8–16 ways that is a lot of state and logic. **Pseudo-LRU = a tournament bracket:** each node only remembers which half was used less recently; follow the arrows to evict, flip them on use. Not always the exact oldest, but close and cheap.

Replacement picks what leaves a full set: LRU is exact but costly; pseudo-LRU is the cheap approximation hardware actually uses.

Why Exact LRU Is Expensive: Counting the Orders

Exact LRU must remember the FULL order

One 4-way set, most → least recently used — some of the possible orders:

A	D	C	B
D	C	B	A
B	A	D	C
C	A	B	D

… 24 orders in total

① count the orders

$$4 \times 3 \times 2 \times 1 = 4! = 24$$

② bits to tell them apart

$$2^4 = 16 < 24 \leq 32 = 2^5$$

③ result

at least 5 bits per set

Pseudo-LRU (previous slide)

- Remembers only "which half is older" at each fork
- 4 ways → 3 forks → 3 bits per set
- N ways → N − 1 bits: grows slowly

State per set grows fast with the number of ways

Ways	Possible orders	Exact LRU (min. bits)	Pseudo-LRU bits
4	24	**5**	**3**
8	40,320	**16**	**7**
16 (e.g., L3)	$\approx 2 \times 10^{13}$	**45**	**15**

It is not just the bits

- The order must be updated on every cache access, within one cycle
- Real exact-LRU designs often keep an age counter per way — even more bits
- So 16-way caches (L3) use an approximation like pseudo-LRU

Exact LRU state grows like **$\log_2(N!)$** bits per set; pseudo-LRU needs only **N − 1** — that is why hardware approximates.

Pseudo-LRU: Initial State and Update Rule

Start: all bits 0 after reset (implementation-defined).
Empty ways are filled first; PLRU picks a victim only when the set is full.

Rule on every access (hit or fill): set the bits on that way's path to point AWAY from it; leave the others.

```
W0: b0←1, b1←1    W1: b0←1, b1←0
W2: b0←0, b2←1    W3: b0←0, b2←0
```

Walk-through from 0 0 0 (red = bit changed by this step, gray = unchanged; read the bits 0 = left, 1 = right)

Step	Event	b0	b1	b2	What changed	Next victim (how it is read)
0	reset	0	0	0	—	**W0 (b0=0 → left, b1=0 → W0)**
1	miss → victim W0, fill it	**1**	**1**	0	W0's path: b0←1, b1←1	**W2 (b0=1 → right, b2=0 → W2)**
2	miss → victim W2, fill it	**0**	1	**1**	W2's path: b0←0, b2←1	**W1 (b0=0 → left, b1=1 → W1)**
3	miss → victim W1, fill it	**1**	**0**	1	W1's path: b0←1, b1←0	**W3 (b0=1 → right, b2=1 → W3)**
4	miss → victim W3, fill it (set full)	**0**	0	**0**	W3's path: b0←0, b2←0	**W0 (b0=0 → left, b1=0 → W0)**
5	hit on W0	**1**	**1**	0	W0's path: b0←1, b1←1	**W2 (not W0 — just used)**
6	miss → victim W2, evict + fill	**0**	1	**1**	W2's path: b0←0, b2←1	**W1 (b0=0 → left, b1=1 → W1)**



Bits flip on every access to point away from the way just used — so the next victim is always somewhere else.

Why "Pseudo"? When PLRU Differs from Exact LRU

Continue from step 4 of the previous slide (filled W0, W2, W1, W3; bits 0 0 0)

Event	Bits b0 b1 b2	True recency (MRU → LRU)	Exact LRU victim	Pseudo-LRU victim
state after step 4	`0 0 0`	W3, W1, W2, W0	**W0**	**W0 ✓ same**
hit on W2	`0 0 1`	W2, W3, W1, W0	**W0**	**W0 ✓ same**
hit on W1	`1 0 1`	W1, W2, W3, W0	**W0 (the oldest)**	**W3 ✗ second-oldest**

Why it picks W3, not W0

- **bit 0 remembers only which HALF was used last:** W1 (left) was just used → it points right
- Inside the left half, W0 is the oldest of all — but no bit stores that
- **Each fork knows "which side is more recent",** not the full order across halves (3 bits instead of 5, p.38)

What pseudo-LRU still guarantees

- **Never the most recently used way (MRU):** its path always points away from it
- Usually an old way — hit rates are typically close to exact LRU
- Only N − 1 bits and a one-cycle update — far cheaper



"Pseudo" = **never the MRU, usually an old line, not always the oldest** — a little accuracy traded for a big cost saving.

Caches on Our DynamIQ SoC: Cortex-A55 vs. Cortex-A76

Level	LITTLE: Cortex-A55	big: Cortex-A76
Core type	in-order, dual-issue	out-of-order, 4-wide decode
L1 I-cache	16–64 KB, 4-way	64 KB, 4-way
L1 D-cache	16–64 KB, 4-way, write-back	64 KB, 4-way, write-back
L2 (private per core)	0 or 64–256 KB, 4-way	256 or 512 KB, 8-way
L3 (DSU, shared)	**up to 4 MB, 16-way — shared by all big and LITTLE cores**	
Line size	64 B	64 B

Table: configurable ranges from the Arm TRMs. Below: what one SoC actually chose, read from sysfs (example output).

```
$ cd /sys/devices/system/cpu        # index2 = L2 here (check index2/level)
$ grep . cpu0/cache/index2/{size,ways_of_associativity} cpu4/cache/index2/{size,ways_of_associativity}
cpu0/cache/index2/size:128K                  # LITTLE (CPU 0)
cpu0/cache/index2/ways_of_associativity:4
cpu4/cache/index2/size:512K                  # big (CPU 4)
cpu4/cache/index2/ways_of_associativity:8
```

1 LITTLE misses more — and stalls

Smaller L1/L2 plus in-order execution: memory-bound code slows down most on LITTLE.

2 L3 is shared by both types

A task moved big ↔ LITTLE can still hit in L3 (next slide).

3 Coherence goes through the DSU

Its snoop filter tracks which core may hold a line (coherence protocol: explained later).

Same cache concepts, different sizes: the core type a task lands on changes its miss rate.

The Cache Needs an Address — But Which One?

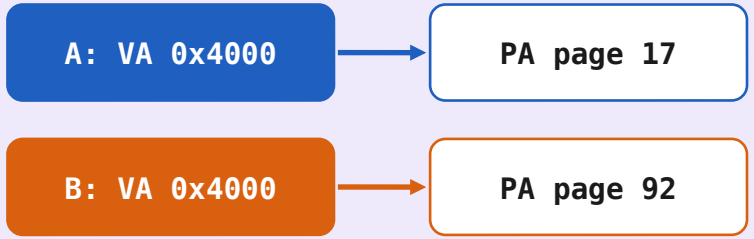
1 So far: split an address



Virtual address (what the program uses) or physical address (where the data really is)?

Answer: physical (PA). Caches are managed by PA. L1 adds a speed trick: it can start the lookup with VA bits while the TLB translates.

2 One VA, many meanings



- Same VA in two processes → different data
- Two VAs can also map to one physical page (shared memory)
- DRAM and other cores only know physical addresses

3 Translate — fast

Walking the page table on every load:

L0 L1 L2 L3

up to 4 extra memory accesses per load

TLB = a cache for translations

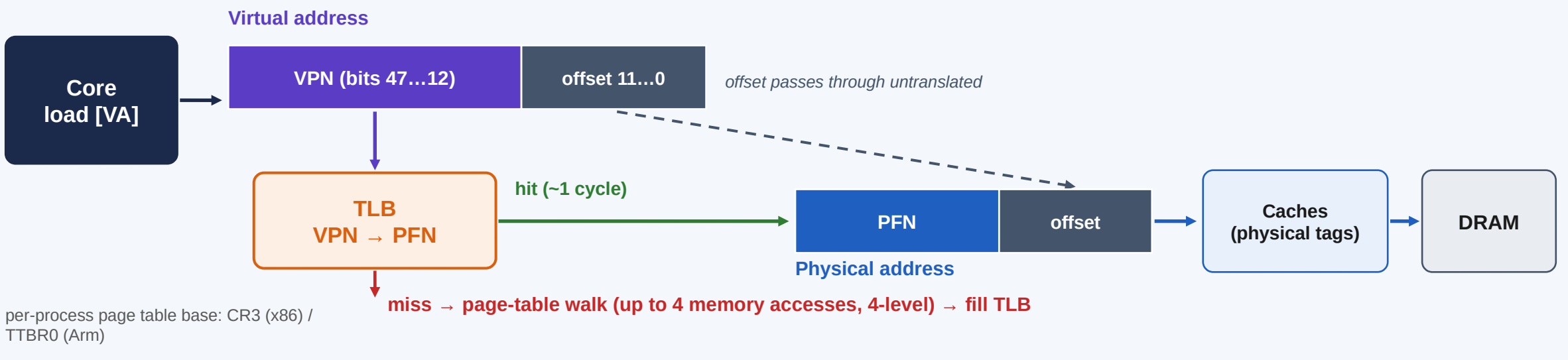
Same ideas as before: sets, ways, hits, misses, replacement.

Basic path



Caches work on physical addresses — **the TLB is the cache in front of the cache.**

Virtual or Physical? The Address Path with the TLB



Who uses which address?

- **Programs and the L1 index:** virtual
- **Cache tags, L2/L3, DRAM, DMA:** physical
- Each process has its own page table → its own VA→PA mapping

TLB sizes on our SoC (Arm TRMs)

TLB	LITTLE: Cortex-A55	big: Cortex-A76
1st-level I-TLB	15 entries, fully assoc.	48 entries, fully assoc.
1st-level D-TLB	16 entries, fully assoc.	48 entries, fully assoc.
2nd-level TLB	1024 entries, 4-way	1280 entries, 5-way

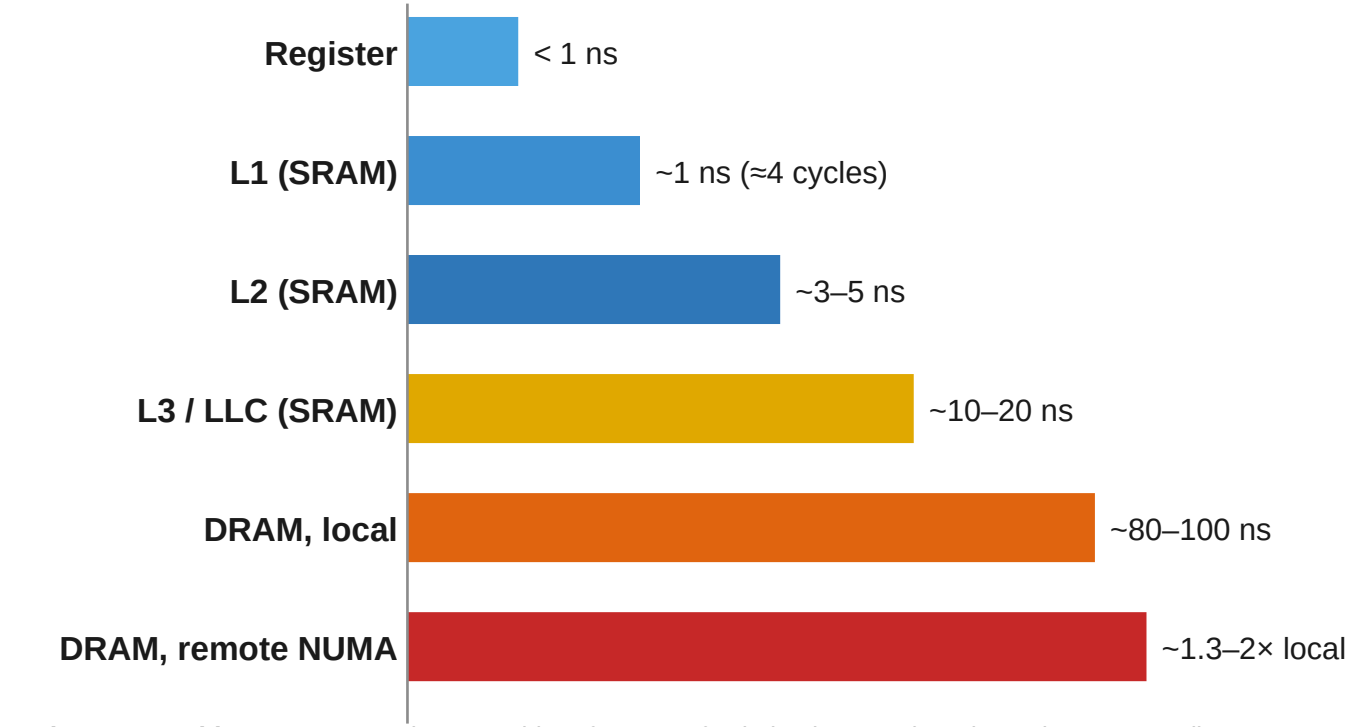
Reach with 4 KB pages: 1280 × 4 KB ≈ 5 MB — huge pages (2 MB) cover far more per entry.

Every memory access needs a translation — the TLB keeps it off the critical path.

Putting It Together: What Does Each Level Cost?

So far: how a line is found, filled, written back and translated (TLB). **Now: what does each level cost — and why should the scheduler care?**

Typical access latency (log scale)



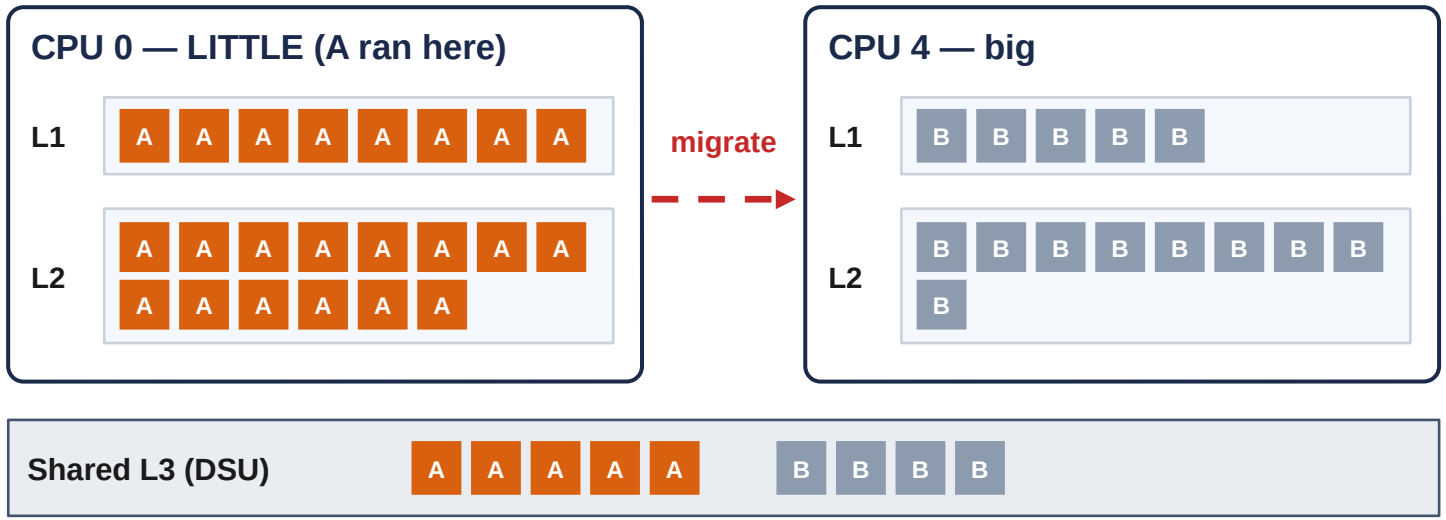
Caches are on-chip SRAM: 6 transistors per bit — fast, no refresh, but large and costly, so they stay small. **DRAM:** 1 transistor + 1 capacitor per bit — dense and cheap, but slower and must be refreshed. *Latencies: orders of magnitude at 3–4 GHz.*

Why this matters for scheduling

- A running task builds its working set in the caches of its core
- After a context switch or migration, that data may be gone
- Refill comes from L3 or **DRAM (~100× slower than L1)**
- One DRAM miss ≈ hundreds of instructions of lost work
- A TLB miss adds a page walk on top (previous slide)

Where a task ran last matters: its data is still **warm** there — next: what happens when it moves.

Warm vs. Cold: What Happens When a Task Moves?



Case 1: A resumes on CPU 0

- L1/L2 hits (~1–4 ns)
- Useful work starts immediately

Case 2: A moves up to big CPU 4

- Private L1/L2 cold → refill
- Shared DSU L3 may still hold A's lines (cheaper than DRAM)

Rough cost of a cold start

- Working set 256 KB = **4,096 lines** × 64 B
- Worst case, serialized DRAM misses: 4,096 × ~90 ns ≈ **0.37 ms**
- Memory-level parallelism hides part of it — still tens to hundreds of µs
- Compare: frame budget **16.7 ms**, audio period ~5–10 ms
- Once is fine; **frequent migrations add up**

Cache affinity: keep a task near the caches that hold its data — unless waiting for that CPU costs more.

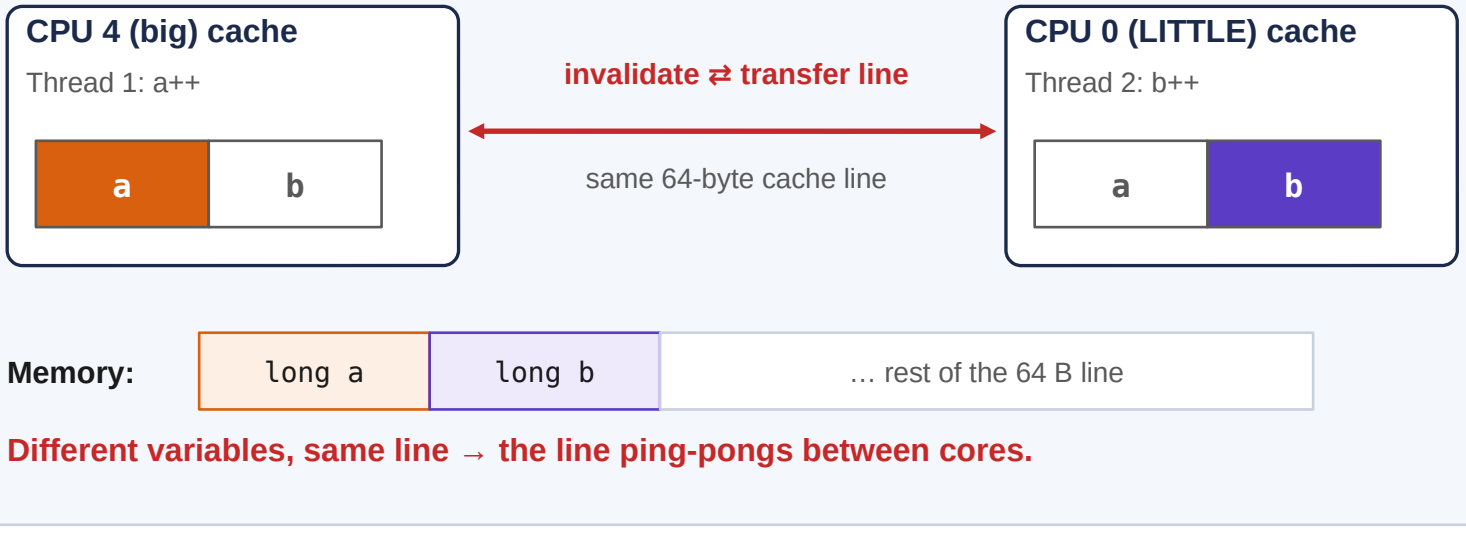
False Sharing: Cache-Line Traffic Without Real Sharing

Why a 64 B line matters

Caches move and track data in 64 B lines. When one core writes a line, the hardware removes (invalidates) the copies in other cores' caches so nobody reads a stale value.

Same data X shared → moving the line is necessary

DIFFERENT variables a, b in one line → pure waste



Different variables, same line → the line ping-pongs between cores.

Bad

```
struct counters {
  long a;  // thread 1
  long b;  // thread 2
};
```

Fixed: one line per writer

```
#include <stdalign.h>
struct counters {
  alignas(64) long a;
  alignas(64) long b;
};
```

CHECKPOINT

Two worker threads increment separate counters in one struct. Adding cores makes it slower. Why?

Answer

- False sharing: both counters sit in one cache line
- Pad/align to 64 B, or keep per-thread counters and sum at the end

False Sharing in Code: Two Threads, Two Counters

```c
#include <pthread.h>
#include <stdio.h>
#define N 200000000L

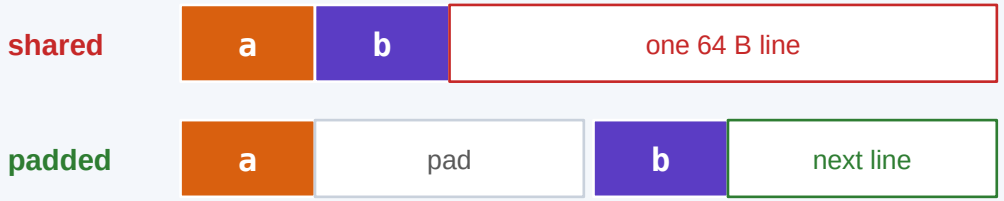
struct counters {
    volatile long a;              // written only by thread 1
#ifdef PAD
    char pad[64 - sizeof(long)];  // push b to the next line
#endif
    volatile long b;              // written only by thread 2
} __attribute__((aligned(64)));
static struct counters c;

void *inc_a(void *p) { for (long i = 0; i < N; i++) c.a++; return 0; }
void *inc_b(void *p) { for (long i = 0; i < N; i++) c.b++; return 0; }

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, inc_a, NULL);
    pthread_create(&t2, NULL, inc_b, NULL);
    pthread_join(t1, NULL);  pthread_join(t2, NULL);
    printf("a=%ld b=%ld\n", c.a, c.b);
}
```

```
$ gcc -O2 -pthread fs.c -o shared
$ gcc -O2 -pthread -DPAD fs.c -o padded
$ time taskset -c 0,4 ./shared
real   0m2.41s          # line ping-pongs
$ time taskset -c 0,4 ./padded
real   0m0.52s          # each core owns its line
$ perf c2c record ./shared; perf c2c report
```

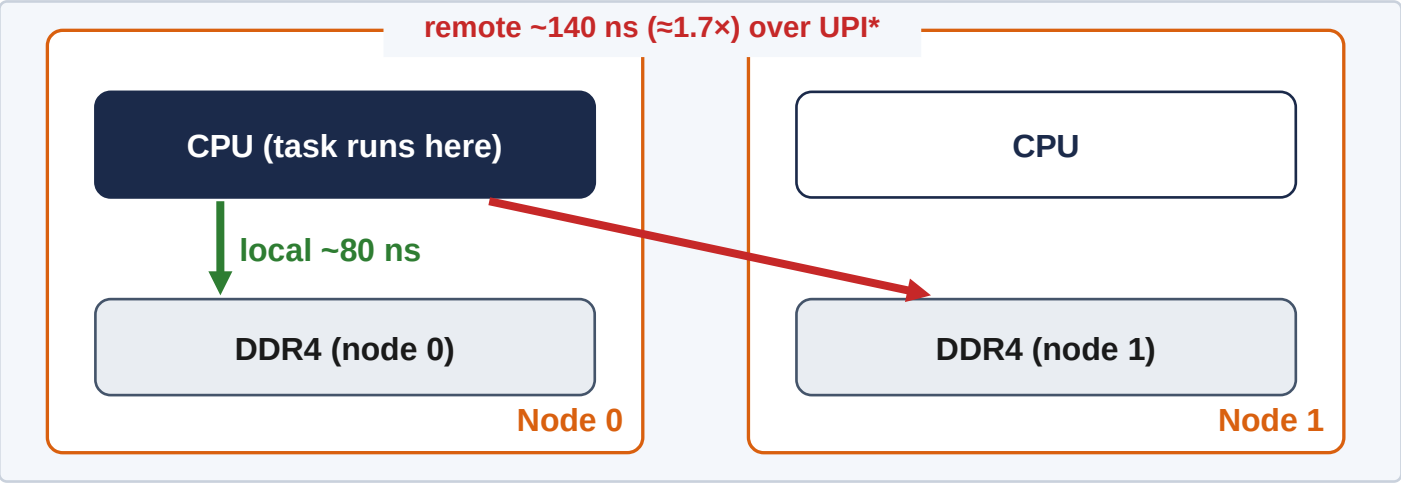
Memory layout



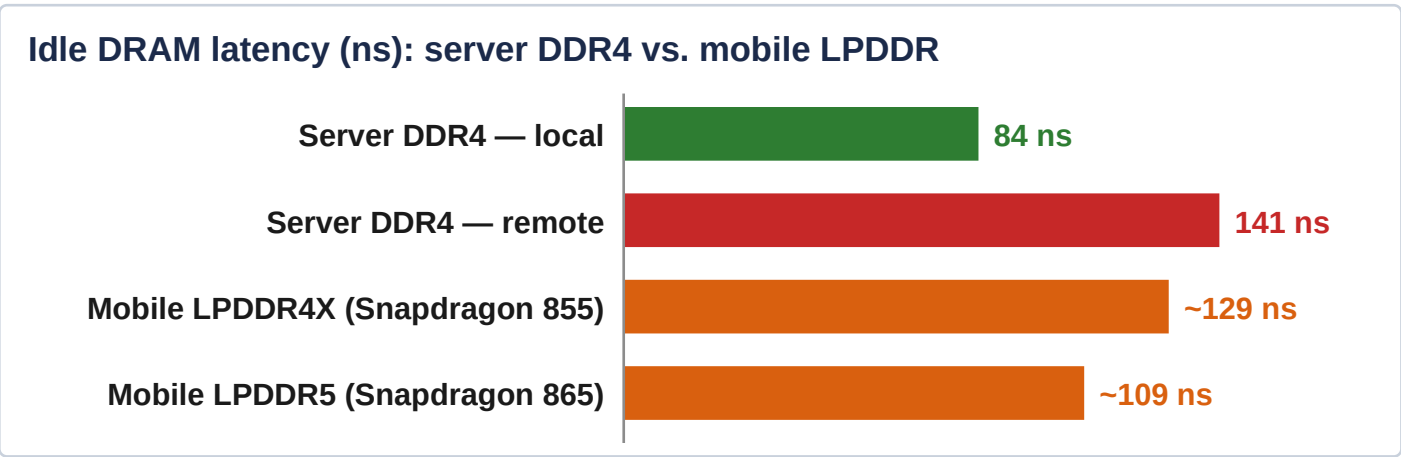
Timings are illustrative — run it on your target. Use two different cores, not SMT siblings.

True sharing = same variable (needs atomics). **False sharing** = different variables, same line — fix by layout, not by locks.

UMA vs. NUMA: Local and Remote Access




```
$ numactl -H | tail -3
node   0   1
  0:  10  21
  1:  21  10
$ ./mlc --latency_matrix
Measuring idle latencies (in ns)...
              Numa node
Numa node       0       1
       0     84.2    141.4
       1    141.5     82.4
```



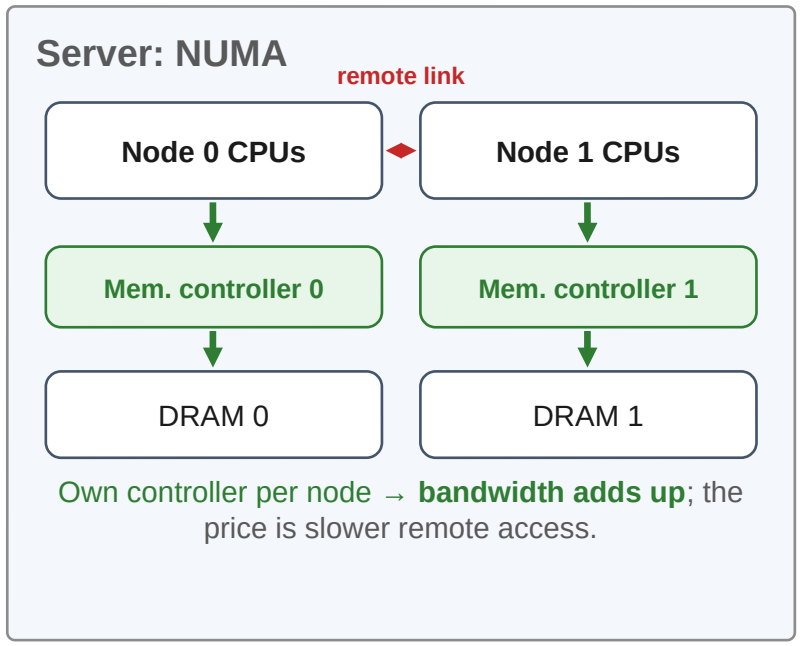
What the numbers say

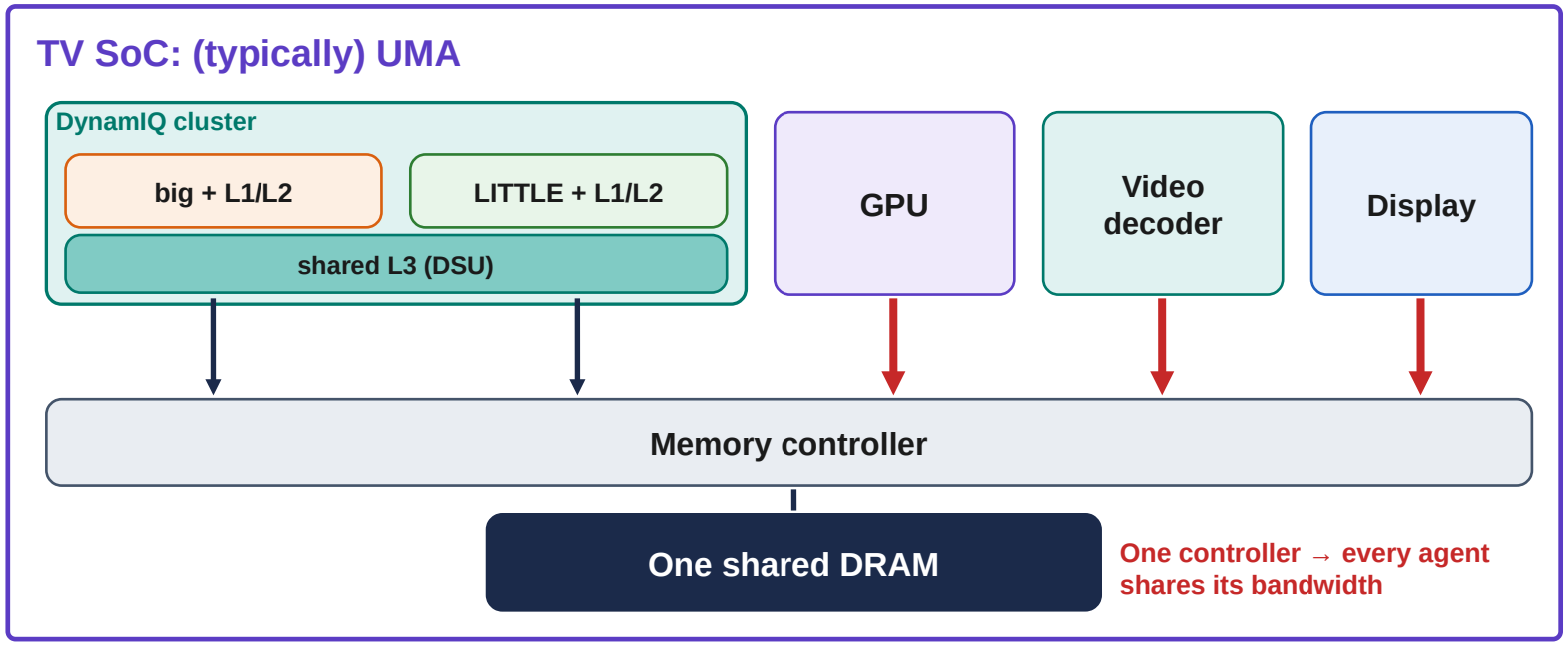
- **Server:** remote ≈ 1.7× latency, ~0.4× bandwidth (79 vs 32 GB/s)
- **Mobile:** one node, no "remote" — but LPDDR trades latency for power
- **A TV/phone DRAM miss ≈ a server's remote access**

** UPI = Intel Ultra Path Interconnect, the socket-to-socket link for remote memory and coherence traffic (AMD: Infinity Fabric). Idle latency; sources: Intel MLC, AnandTech.*

On NUMA, run close to your memory; on LPDDR SoCs, every DRAM miss is already "remote-class" — keep data in cache.

Does NUMA Matter for webOS?





1 One memory node

numactl -H typically shows "available: 1 nodes" — NUMA policies rarely apply (next slide).

2 Locality still exists

big and LITTLE share L3, but each core has private L1/L2: moving a task is a "mini remote access".

3 Bandwidth is shared

CPU, GPU, video decoder and display all compete for the same DRAM.

For webOS, think **cache locality (private L1/L2) + shared memory bandwidth** instead of NUMA nodes.

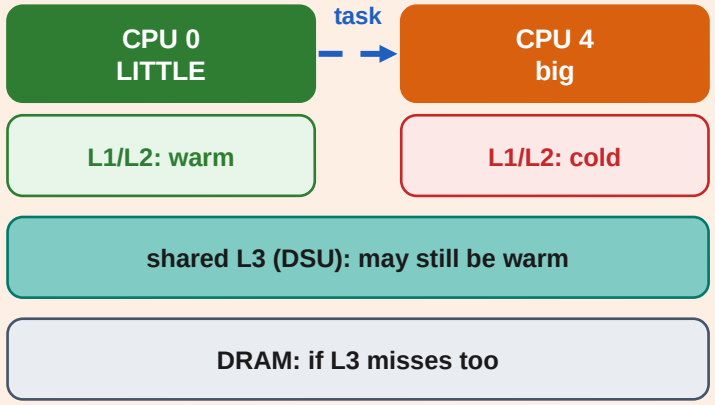
On Our UMA SoC: Three Things to Check Instead of NUMA

1 One memory node

```
$ numactl -H
available: 1 nodes (0)
node 0 cpus: 0 1 2 3 4 5
node 0 size: 3840 MB  # kernel-managed RAM
node 0 free: 1210 MB  # unused right now
node distances:
node   0
  0:  10
```

- **size 3840 MB:** RAM the kernel manages. 4 GB minus areas reserved before Linux starts (firmware, GPU, video decoder, security) — TV SoCs often reserve a lot for video
- **free 1210 MB:** memory nobody uses right now
- **One node, all CPUs:** NUMA policies (--membind) rarely matter

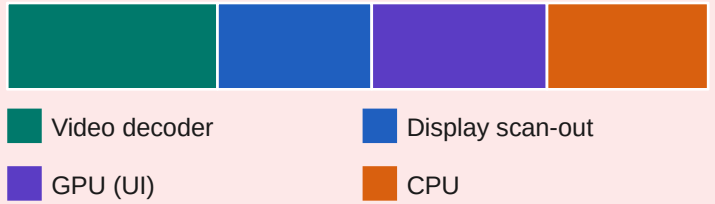
2 Locality still exists



- Refill from L3: tens of ns per line; from DRAM: ~100 ns (p.44)
- So moving a task is a "mini remote access" (p.45)
- Same rule as NUMA, smaller scale: keep a task near its data

3 Bandwidth is shared

One DRAM, many readers during 4K playback (illustrative)



4K 60 Hz scan-out alone:
3840 × 2160 × 4 B × 60 ≈ 2 GB/s

- Heavy decode + display traffic → the CPU's DRAM misses wait longer (loaded latency > idle latency)
- Another reason to keep CPU data in cache

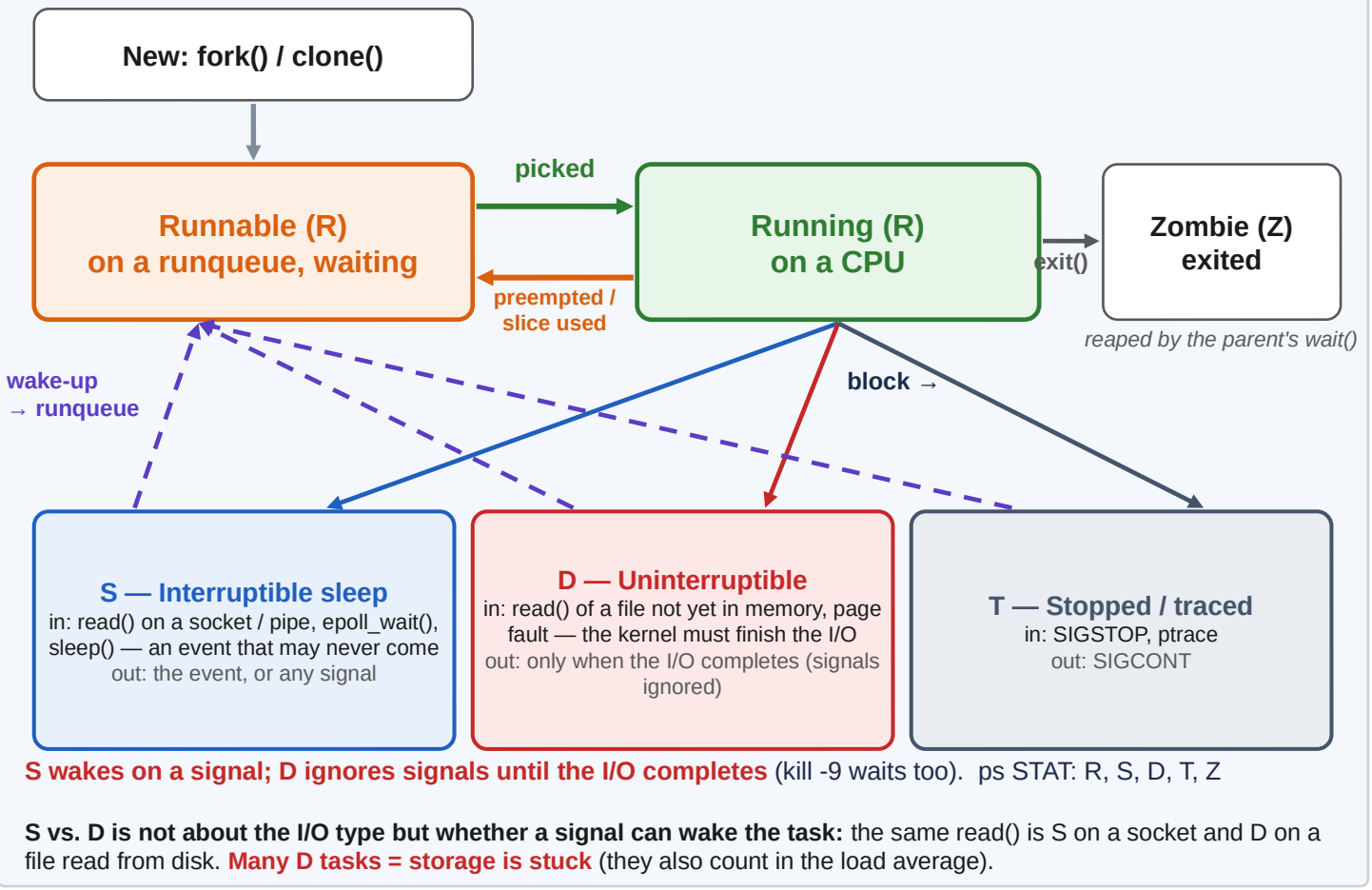
On a UMA TV SoC, check **one node, cache locality, shared bandwidth** — not NUMA placement.

PART 2: CPU Performance and Scheduling

- **Week 4**

- CPU Topology and big.LITTLE
  - What does the hardware look like to the scheduler?
- Our SoC: Caches and the Memory Hierarchy
  - Where does data live, and what does moving a task cost?
- **Linux Process Scheduling**
  - How does Linux organize runnable tasks and place them on CPUs?
- Scheduling in webOS
  - How can webOS keep UI and media smooth under load?

What Is Scheduling? Task States and Runqueues



S wakes on a signal; D ignores signals until the I/O completes (kill -9 waits too). ps STAT: R, S, D, T, Z

S vs. D is not about the I/O type but whether a signal can wake the task: the same read() is S on a socket and D on a file read from disk. **Many D tasks = storage is stuck** (they also count in the load average).

Linux view

- **`TASK_RUNNING`** = runnable or running (R)
- **`TASK_INTERRUPTIBLE`** (S), **`TASK_UNINTERRUPTIBLE`** (D)
- Each CPU has its own runqueue (`struct rq`)
- Sleeping tasks sit on wait queues, not on runqueues
- Scheduler job: pick the next task from the runqueue

Example: a blocking socket read()

read() on a socket with no data → S on a wait queue → data arrives and wakes the task → back on a runqueue → the scheduler decides when it runs again

A wake-up only makes a task runnable — the scheduler decides when (and where) it actually runs.

Scheduling Classes: Who Is Considered First?

Scheduling class hierarchy — highest priority first

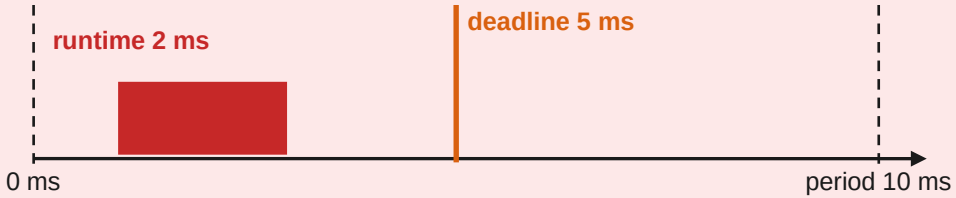
high

stop	kernel-only: CPU hotplug, migration work
deadline	SCHED_DEADLINE (runtime / deadline / period)
rt	real-time: SCHED_FIFO, SCHED_RR (priority 1–99, higher wins)
fair	SCHED_NORMAL (OTHER), SCHED_BATCH (nice −20…19)
idle	SCHED_IDLE (only when nothing else)

low

pick_next_task() checks the classes from top to bottom; the first class with a runnable task wins.

SCHED_DEADLINE in one picture



Needs 2 ms of every 10 ms, done within 5 ms → **2 / 10 = 20% of a CPU**

Admission control: the kernel adds up runtime / period of all deadline tasks; the sum must stay under a limit (default 95%).



20% + 30% + 50% = 100% > 95% → `sched_setattr()` fails with **EBUSY**

```
$ chrt -p 1234
pid 1234's current scheduling policy: SCHED_OTHER
pid 1234's current scheduling priority: 0
$ ps -eLo tid,cls,rtprio,ni,comm | head -3
  TID CLS RTPRIO  NI COMMAND
  812  FF     50   - audio_out
 2210  TS      -   0 renderer
# FF = SCHED_FIFO (rt class)   TS = time-sharing (fair)
```

A runnable RT task always runs before any fair task — use RT only for short, bounded work.

Two Steps: Pick a Class, Then a Task Inside It

Common view: "line up every task and compare one priority number." **Linux: first pick a class, then pick a task inside that class.**

`pick_next_task()`

① which class? (top → bottom)

deadline → rt → fair → idle

first class with a runnable task wins

② which task inside it?

deadline	earliest deadline first
rt	highest priority 1–99 (FIFO / RR)
fair	CFS / EEVDF, weighted by nice -20…19
idle	only when nothing else runs

Three different comparisons

Comparison	Kind	Winner
SCHED_FIFO 1 vs. NORMAL nice -20	between classes	FIFO 1 — always
RT priority 80 vs. RT 50	inside rt	priority 80
nice 0 vs. nice 10	inside fair	both run; nice 0 gets more CPU

Example: all three are runnable

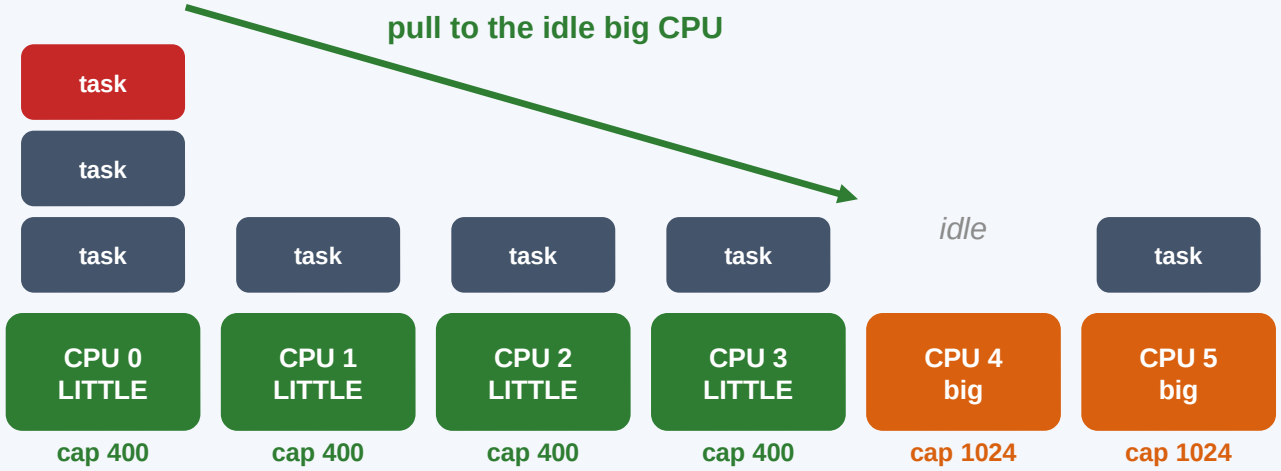
`audio_out`	FIFO 50	① runs first (rt class)
`renderer`	nice 0	② then fair: larger share
`logger`	nice 10	② fair: smaller share

Class first, then task. An RT task that never sleeps can starve every fair task — keep RT work short and bounded.

Per-CPU Runqueues and Load Balancing on big.LITTLE

One runqueue per CPU on our SoC

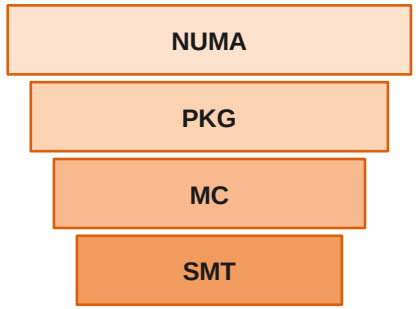
Compare load to capacity, not task count: 2 tasks on a big core (1024) may be fine; the same 2 on a LITTLE core (400) can be overloaded.



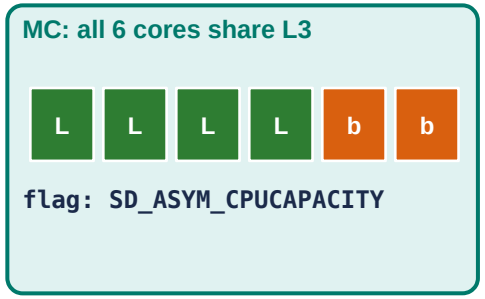
Illustrative capacities and queue lengths.

Scheduling domains: server vs. our SoC

Server



Our DynamIQ SoC



No SMT, PKG or NUMA levels: one balancing domain, but cores of two sizes.

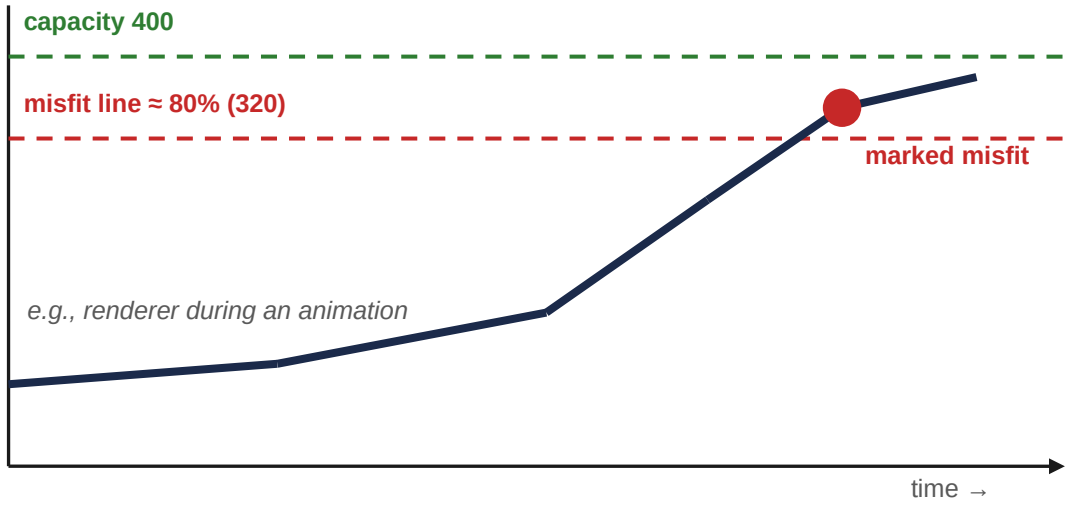
Balancing triggers

- Periodic: from the scheduler tick
- Newly idle: a CPU about to idle pulls work
- Wake-up: choose a good CPU up front (EAS)
- Misfit: a task too big for its LITTLE core (next slide)

On big.LITTLE, balance **capacity**, not task count. `($ ls /sys/kernel/debug/sched/domains/cpu0/)`

Misfit Tasks: When a Task Outgrows Its LITTLE Core

Task util on LITTLE CPU 0 (capacity 400)



util > 80% of capacity → next balance: pull to big → running? active balance moves it

uclamp.min can make a task count as misfit sooner.

Two regimes on big.LITTLE

Not overutilized: every CPU < 80% of capacity

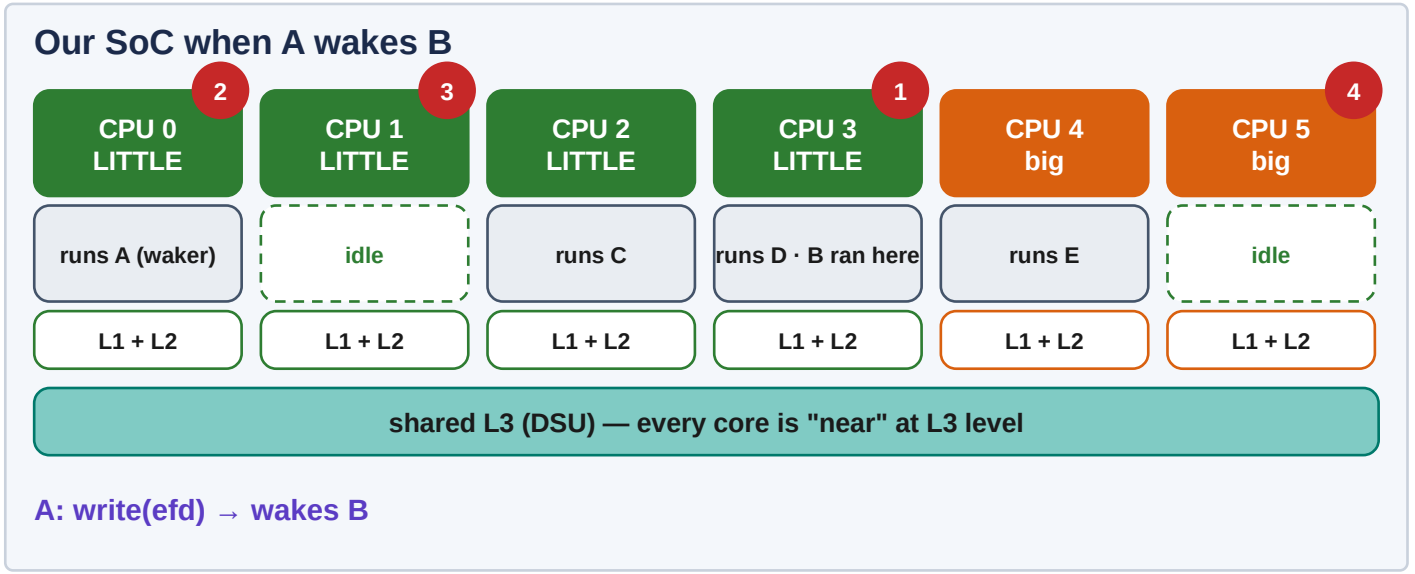
- EAS places tasks at wake-up for lowest energy
- Periodic load balancing is mostly skipped
- Light tasks stay on LITTLE

Overutilized: some CPU > 80% of capacity

- EAS steps aside — performance first
- Capacity-aware load balancing spreads work
- big cores take the heavy tasks first

Misfit tasks move up to big; **EAS keeps the rest on LITTLE** until the system gets busy — then plain capacity-aware balancing takes over.

Wake-up Placement on big.LITTLE: Where Should B Run?



#	Candidate	Good	Bad
1	prev CPU 3	B's private L1/L2 may be warm	busy with D → B waits
2	near waker (wake_affine)	A's fresh data is close	CPU 0 is running A
3	idle CPU 1 (select_idle_sibling)	runs now; same L3	B's own private cache lost
4	idle big CPU 5	fastest core	more energy — only if B needs it

big.LITTLE adds capacity and energy (EAS, when not overutilized)



util 200 (fits LITTLE) → CPU 1

util 600 (renderer) → big CPU 5

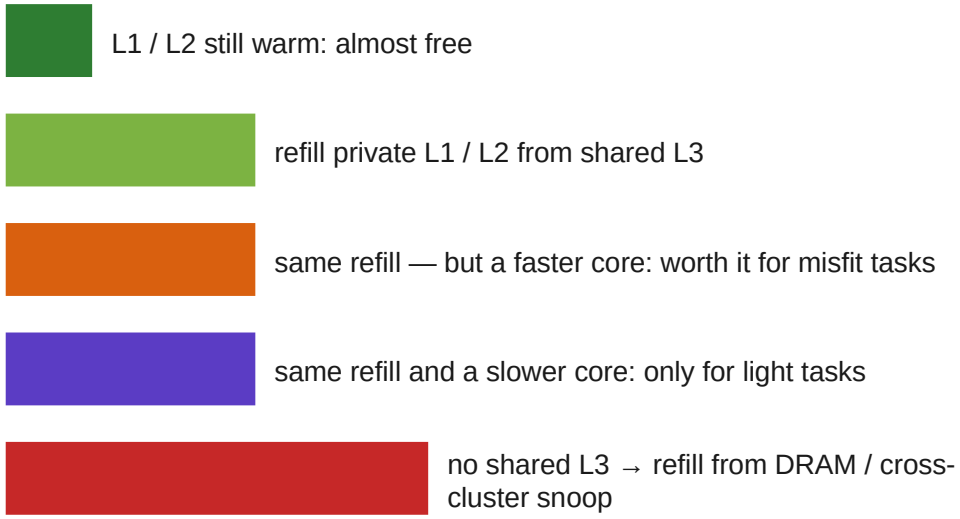
Kernel path: `try_to_wake_up() → select_task_rq_fair() → find_energy_efficient_cpu()  (else wake_affine() + select_idle_sibling())`

Good wake-up placement means less load balancing later. Illustrative values.

At wake-up the scheduler trades **cache warmth against waiting time** — and on big.LITTLE also **capacity and energy**.

Cache Affinity and Migration Cost

Migration cost on our DynamIQ SoC



Same core (resume)	L1 / L2 still warm: almost free
Other LITTLE core	refill private L1 / L2 from shared L3
LITTLE → big (up)	same refill — but a faster core: worth it for misfit tasks
big → LITTLE (down)	same refill and a slower core: only for light tasks
Classic big.LITTLE: other cluster	no shared L3 → refill from DRAM / cross-cluster snoop

How Linux protects cache-hot tasks

- `task_hot()`: ran within the last **migration_cost** (default 0.5 ms) → avoid moving
- Tunable: `/sys/kernel/debug/sched/migration_cost_ns`
- Repeated balance failures eventually force a migration

Too sticky

Idle CPUs while tasks queue elsewhere

Too eager

Tasks bounce, caches always cold

Balance point

Move when the wait saved is larger than the cache lost. On big.LITTLE, an up-migration also buys capacity — a misfit task pays it back.

Cache affinity is a cost model: on our SoC every move refills L1/L2 from L3 — and changes the core size too.

CPU Affinity API: sched_setaffinity() and taskset

```c
#define _GNU_SOURCE
#include <sched.h>
#include <pthread.h>

cpu_set_t set;
CPU_ZERO(&set);
CPU_SET(2, &set);                    // allow CPU 2
CPU_SET(3, &set);                    // ... and CPU 3
sched_setaffinity(0, sizeof(set), &set);
                                     // 0 = calling thread

// per pthread
pthread_setaffinity_np(tid, sizeof(set), &set);

// how many CPUs may I use?
sched_getaffinity(0, sizeof(set), &set);
int n = CPU_COUNT(&set);
```

```
$ taskset -c 2,3 ./decoder      # LITTLE CPUs 2-3
$ taskset -pc 1234              # show
pid 1234's current affinity list: 0-5
$ taskset -pc 4-5 1234          # big CPUs 4-5 (here!)
$ grep . /sys/devices/system/cpu/cpu*/cpu_capacity
```

Pros

- Keeps caches warm
- Isolates noisy or critical work
- Forces a core type (big / LITTLE)

Cons

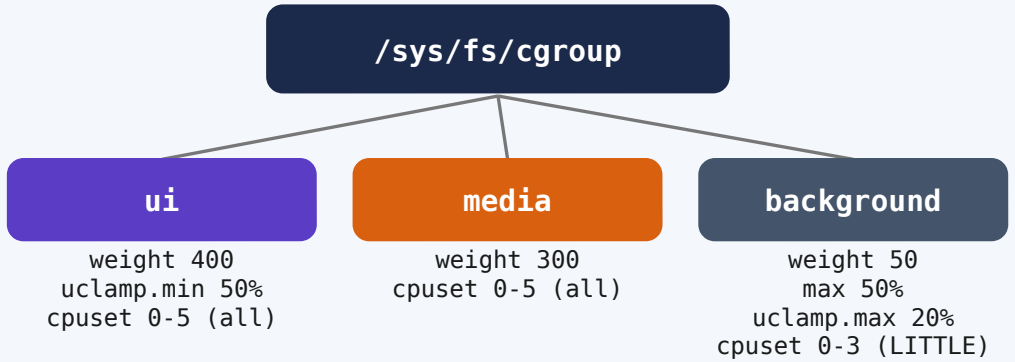
- No balancing for that task
- Pinned to LITTLE: no misfit up-move
- Pinned to big: EAS energy savings lost
- CPU IDs / types differ per SoC

Affinity is a **constraint**, not a hint — prefer cgroups / cpusets for groups of tasks.

cgroups v2: Policy for Groups, Not Single Tasks

Affinity (previous slide): one task at a time. **cgroup:** a whole group of processes/threads — no code changes.

webOS example (illustrative)




```
# cd /sys/fs/cgroup && echo "+cpu +cpuset" > cgroup.subtree_control
# mkdir background && cd background
# echo 50 > cpu.weight;  echo "50000 100000" > cpu.max
# echo 0-3 > cpuset.cpus; echo 20 > cpu.uclamp.max
# echo 1234 > cgroup.procs
```

Four knobs, four different questions

File	Question it answers	Our SoC example
cpu.weight	When groups compete, what share does each get?	ui 400 : media 300 : bg 50
cpu.max	At most how much CPU, even if idle CPUs exist?	bg 50000 100000 = 0.5 CPU
cpuset.cpus	Which CPUs may it run on? (= core type here)	bg 0-3 = LITTLE only
cpu.uclamp.min / max	How "big" should the scheduler and cpufreq think it is?	ui min 50% · bg max 20%

Also read-only cpu.pressure (PSI): how long tasks waited for CPU.

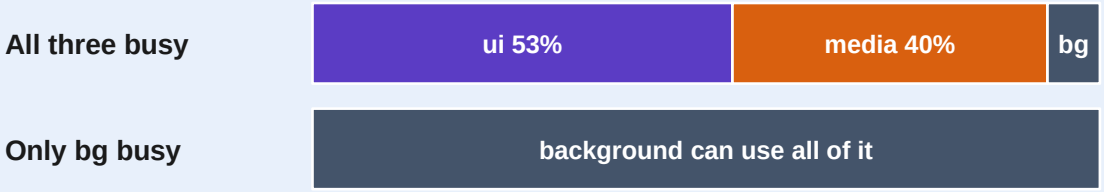
systemd equivalents

- `CPUWeight=50  CPUQuota=50%  AllowedCPUs=0-3`

cgroups apply scheduling policy to whole groups — UI, media, background — without touching their code.

weight vs. max, and Combining All Four Knobs

`cpu.weight` — relative share, only when competing



Not an absolute amount: idle CPU time is not wasted.

Question: "when we compete, how much do I get?"

`cpu.max = 50000 100000` — absolute cap



every 100 ms period: at most 50 ms of CPU ≈ 0.5 CPU

Even if CPUs are idle, it cannot use more.

Question: "no matter what, never more than this."

Example: background firmware updater on our SoC — combine all four

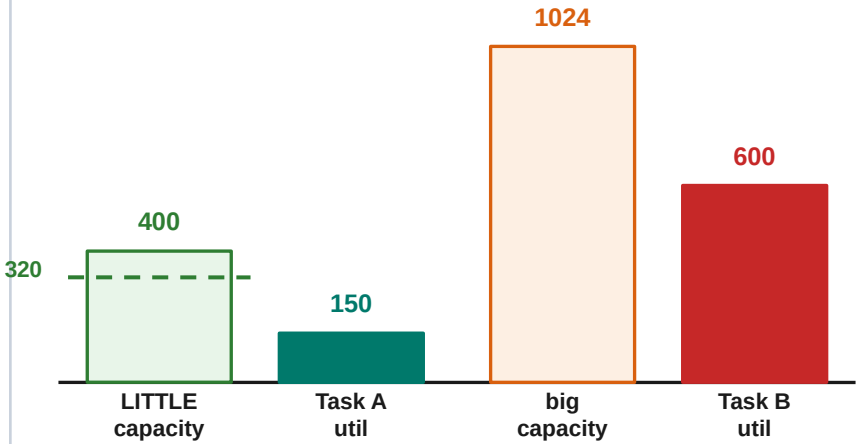
`cpuset.cpus = 0-3`	`cpu.weight = 50`	`cpu.max = 20–50%`	`cpu.uclamp.max = 20%`
→ cannot reach the big cores	→ yields to UI when they compete	→ cannot use too much CPU at all	→ no high frequency / big-core demand

Result: the update progresses slowly on LITTLE, at low frequency — and the UI keeps the big cores. **This is "LITTLE, capped" (p.21).**

weight = share when competing; max = hard ceiling; **cpuset = where (core type); uclamp = how big it looks.**

EAS in Action: Clamped Util + Capacity + Energy

① Does the task fit? (util ≤ 80% of capacity)



② Among CPUs that fit, the cheapest energy

Task	Candidate	Fits?	Est. energy
A (150)	idle LITTLE CPU 1	✓	**1.0× → pick**
A (150)	idle big CPU 5	✓	**≈ 2×**
B (600)	any LITTLE	✗	—
B (600)	idle big CPU 5	✓	**only choice**

Relative energy from the p.13 curves (illustrative).

How EAS decides

- **Inputs:** util (PELT, clamped by uclamp) + cpu_capacity + energy model (power per OPP, from the device tree)
- For each CPU where the task fits: estimate the energy if it ran there → pick the lowest (only when not overutilized, p.56)

Setting a clamp from code (per task)

```
struct sched_attr a = {
  .size           = sizeof(a),                // struct version for the kernel
  .sched_policy   = SCHED_OTHER,              // stay in the fair class
  .sched_flags    = SCHED_FLAG_UTIL_CLAMP_MIN, // change uclamp.min only
  .sched_util_min = 512 };                    // 0..1024: treat me as >= 512
syscall(SYS_sched_setattr,  // the system call; older glibc had no wrapper
        0,    // target thread ID: 0 = the calling thread (else its TID)
        &a,   // the settings above
        0);   // flags: reserved, always 0
```

Effect: real util 150 → seen as 512 > 320 → **big core + higher frequency**

Same clamp, other ways: `uclampset -m 512 -p <pid>`; per group: cpu.uclamp.min (p.60)

EAS decides with the **clamped** util — so cgroup / uclamp settings directly change which core a task lands on.

Observing Scheduling

1 perf sched: who waited, how long?

```
$ sudo perf sched record -a -- sleep 5
$ sudo perf sched latency --sort max
 Task           | Runtime ms | Switches | Max delay ms
 renderer:2210 |    812.4   |   1450   |   18.73
 lsm:1033       |    402.1   |    980   |    2.10
```

Reading: renderer ran 812 ms, switched 1450 times, and once waited **18.73 ms** from runnable to running — more than the 16.7 ms frame budget at 60 FPS → a frame can be missed.

2 /proc/<pid>/sched: cumulative state of this task

```
$ cat /proc/2210/sched
se.sum_exec_runtime      :    812.41
nr_voluntary_switches    :      1302
nr_involuntary_switches :       148
policy                   :         0
```

3 ps: where is it running now?

```
$ ps -eLo tid,cls,rtprio,ni,psr,pcpu,comm \
    --sort=-pcpu | head -3
  TID CLS RTPRIO NI PSR %CPU COMMAND
 3120  TS      -  0   5 97.0 indexer   # PSR 5 = big
 2210  TS      -  0   5 41.2 renderer
```

Reading: PSR 5 = CPU 5, a big core. The background indexer and the latency-sensitive renderer may be competing for the same big core. **Caveat: ps is a snapshot** — use perf sched / ftrace for how long.

4 ftrace: which events, in what order?

```
$ sudo trace-cmd record -e sched:sched_switch \
      -e sched:sched_wakeup -e sched:sched_migrate_task sleep 3
$ trace-cmd report | grep renderer | head
  # or open the trace in Perfetto / KernelShark
```



Suggested fix: the background indexer holds big CPU 5 and raises the renderer's latency (18.7 ms > 16.7 ms) → restrict the indexer to a LITTLE cpuset (`cpuset.cpus = 0-3`, p.61). Check core types with `cpu_capacity`.

ps = where it runs · **/proc/<pid>/sched** = cumulative state · **perf sched** = how long it waited · **ftrace** = which events, in what order — diagnosing affinity, cpusets and big.LITTLE scheduling on a real system.

ftrace in Practice: From trace.dat to a Timeline

1 Record → trace.dat

```
$ sudo trace-cmd record -e sched:sched_switch -e sched:sched_wakeup \
      -e sched:sched_migrate_task sleep 3
```

→ events saved to **`./trace.dat`** (binary, current dir; -o to rename). report reads it by default.

3 Visualize: what KernelShark / Perfetto show



`kernelshark trace.dat` or `trace-cmd report > trace.txt` → ui.perfetto.dev

2 Read: `trace-cmd report` — each line: task-PID [CPU] time: event: details

```
$ trace-cmd report | grep -E "renderer|indexer"
 renderer-2210 [000] 1523.380210: sched_migrate_task: comm=renderer pid=2210 orig_cpu=0 dest_cpu=5        # ①
 renderer-2210 [005] 1523.383120: sched_wakeup:       comm=renderer pid=2210 prio=120 target_cpu=005     # ②
 indexer-3120 [005] 1523.402031: sched_switch:       prev_comm=indexer prev_state=R ==> next_comm=renderer  # ③
```

① renderer moved to big CPU 5. ② it became runnable at .383120. ③ it got the CPU only at .402031 — **waited ≈ 18.9 ms**; `prev_state`=R = indexer was preempted, not sleeping.

perf sched = statistics: how long did it wait? → **ftrace** = every event in order: why, and because of whom? Use them in this order.

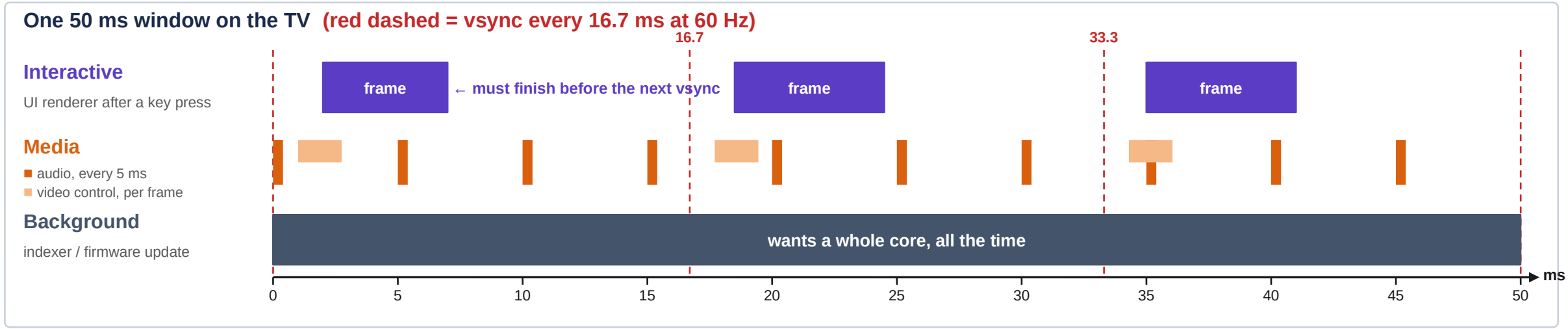
trace.dat keeps every scheduling event in order — **wakeup → switch tells you exactly who held the CPU** while your task waited.

PART 2: CPU Performance and Scheduling

- **Week 4**

- CPU Topology and big.LITTLE
  - What does the hardware look like to the scheduler?
- Our SoC: Caches and the Memory Hierarchy
  - Where does data live, and what does moving a task cost?
- Linux Process Scheduling
  - How does Linux organize runnable tasks and place them on CPUs?
- **Scheduling in webOS**
  - How can webOS keep UI and media smooth under load?

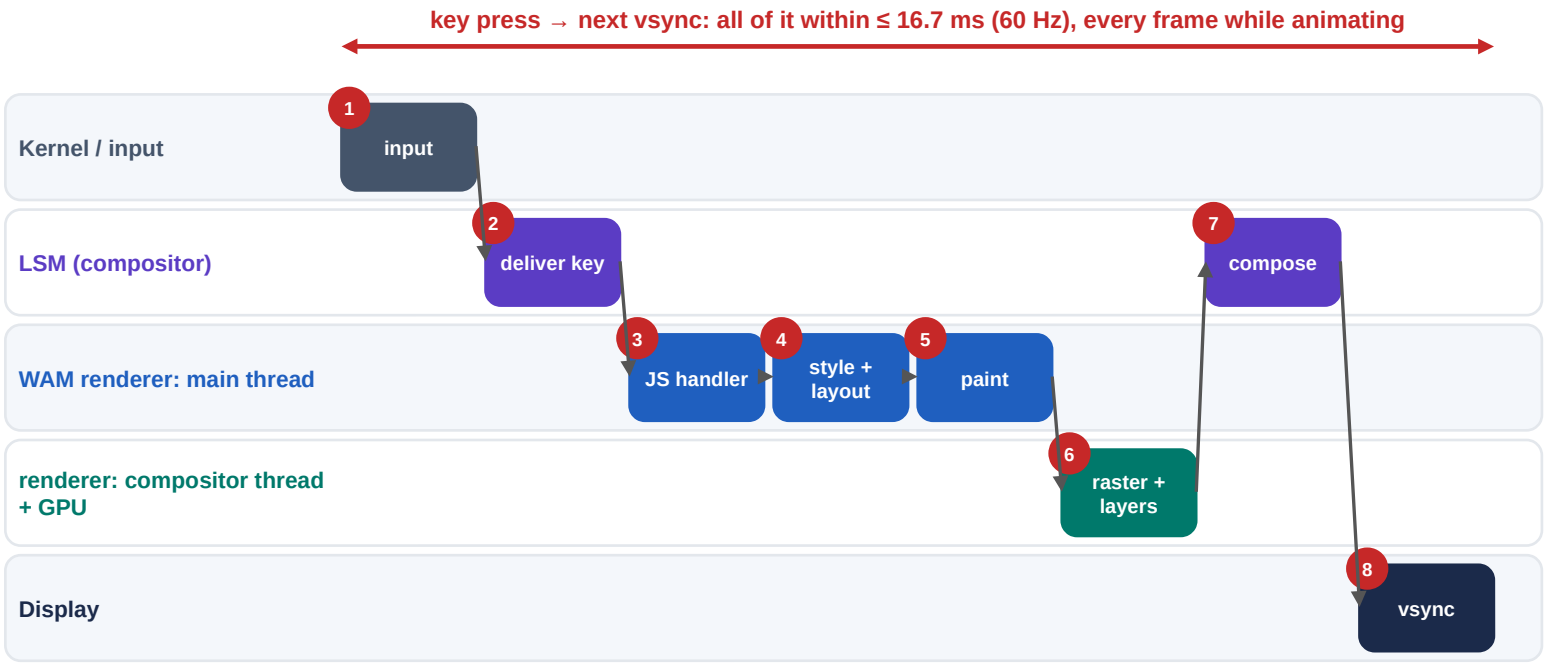
Three Kinds of Work, Running at the Same Time



	Interactive	Media	Background
Examples	LSM compositor, WAM renderer, input	demux, A/V sync, audio output	app / firmware update, indexing, logging
What matters	**latency**	**periodic deadlines**	**throughput, low power**
Time scale	ms after the user acts; ≤ 16.7 ms / frame	every 5 ms (audio), every frame (video)	seconds to minutes; can wait
CPU demand	bursty: idle, then a sharp peak	small, regular, predictable	steady, as much as it gets
Failure looks like	jank, sluggish remote	audio glitch, dropped frame	nothing — until it steals CPU from the others
On our SoC	big for bursts (EAS / misfit, uclamp.min)	LITTLE is enough; pixels go to the HW decoder	LITTLE, capped (cpuset, uclamp.max, weight)

One TV, three requirements at once — the scheduler must serve latency, deadlines and throughput together, on shared cores, caches and DRAM.

From a Key Press to Pixels: What the UI Renderer Does



What happens at each step

Example: press → on the remote to move the focus to the next app tile — the most common TV action.

1. Remote signal → kernel input driver creates an input event
2. LSM (Wayland compositor) sends the key to the focused app (WAM)
3. Renderer main thread runs the app's JS keydown handler → DOM/CSS change
4. Style + layout: recompute styles and positions of what changed
5. Paint: record the draw commands (what to draw, not pixels yet)
6. Compositor thread + GPU: raster to pixels, combine layers → app frame
7. LSM composes the app frame with system UI and video
8. Display shows the final frame at the next vsync

Who is who

- **WAM**: runs web apps on Chromium (Blink)
- **UI renderer**: the app's Chromium renderer — HTML/CSS/JS → app frame
- **LSM**: Luna Surface Manager (Qt Wayland compositor) — final screen

Why the scheduler matters here

- Idle until the user acts, then a sharp burst
- Waiting in a runqueue (p.4) → step 8 misses vsync → jank
- On our SoC: reach big cores fast (EAS / misfit, uclamp.min)

The "UI renderer" is not one function but a chain of threads — **every link must get a CPU in time, every frame.**

Mapping webOS Components to Scheduling Policies

Component	Workload nature	Candidate policy	Where on our SoC	Why
Audio output thread	periodic, very short	SCHED_FIFO (low RT prio)	**any** LITTLE is enough	buffer underrun is audible
Video pipeline control	periodic, per frame	high weight + uclamp.min (or low RT)	**any** pixels go to the HW decoder	late frames are dropped
Compositor (LSM)	per-frame burst	high weight + uclamp.min	**big** during animation (EAS / misfit)	every frame passes through it
Foreground web app (WAM / Chromium)	bursty, interactive	fg cgroup: weight 400, uclamp.min 50%	**big** for bursts; may use all cores	UI responsiveness
System services (SAM, LS2 services)	request / reply	normal; critical ones ≥ caller	**any**	avoid IPC priority inversion
Updates, indexing, logging	throughput, deferrable	bg cgroup: weight 50, cpu.max, uclamp.max 20%	**LITTLE** cpuset 0-3 only	must not steal big cores / UI time

big CPU 4–5: latency-critical bursts

LITTLE CPU 0–3: light or background work

any let EAS decide (usually LITTLE)



Give each class of work the weakest policy that still meets its deadline.

Reading the Mapping: Four Design Choices

1 RT only for audio

- **The work is tiny and exactly periodic:** e.g., 0.3 ms every 5 ms ≈ 6% of a core
- So a real-time priority cannot starve anyone else
- **Rule from p.53–54:** RT only for short, bounded work

2 UI: weight + uclamp.min, not RT

- **UI bursts are large and vary in length:** as RT they could starve every fair task
- **uclamp.min** instead: big core and high frequency from the very first ms (p.62)
- weight 400 wins CPU share when groups compete

3 "any" = let EAS decide

- Not "don't care": the scheduler picks the cheapest core that fits
- Small, periodic work usually lands on LITTLE
- No pinning → no stuck tasks when a core is busy

4 Services ≥ caller: avoid priority inversion

- **UI → request → SAM;** if SAM runs at a lower priority, it waits behind background work…
- **…and the UI waits for SAM:** a high-priority task stuck behind a low one
- **Fix:** critical services never below their callers

Strong policies cost others: **RT only where the work is tiny and periodic**; everything else gets weight, uclamp and cpusets.

Case Study: Why Does the App Launch Slowly?

1 Record while launching the app

```
$ sudo perf sched record -a -- sleep 10
  # meanwhile: launch the app from Home
# → ./perf.data  (also note: launch took ~1.9 s)
$ cat /proc/pressure/io     # I/O pressure (PSI)
some avg10=62.10 avg60=40.32 ...
full avg10=35.40 avg60=21.08 ...
```

Reproduce and record: launch the app while tracing. I/O pressure is high — tasks often stall waiting for storage.

2 Is the victim waiting for a CPU?

```
$ sudo perf sched timehist -p 2410 --state | sort -k4 -n | tail -2
  time   cpu  task          wait time  sch delay  run time  state
 812.104 [4]  WebAppMgr[2410]  180.233     0.041     2.100    D
 813.402 [4]  WebAppMgr[2410]  142.870     0.038     1.884    D
# sch delay ≈ 0.04 ms, but it slept 140–180 ms at a time, in D
```

Not a CPU problem: sch delay is tiny, but the task sleeps for long stretches in **D** — waiting for eMMC reads and page faults (p.52).

3 Find the culprit: who holds the storage?

```
$ ps -eLo tid,stat,wchan:22,comm | awk '$2 ~ /D/'
 2410 D    folio_wait_bit_common  WebAppMgr
 3120 D    io_schedule            indexer
$ grep -E "read_bytes|write_bytes" /proc/3120/io
read_bytes: 1843200000     write_bytes: 912400000
```

The background indexer is hammering the eMMC: the app's reads queue behind it — an I/O priority inversion (p.69 ④).

4 Fix the I/O policy, then re-measure

```
# B=/sys/fs/cgroup/background; echo "+io" > /sys/fs/cgroup/cgroup.subtree_control
# echo "179:0 rbps=10485760 wbps=10485760" > $B/io.max  # 10 MB/s
# echo 3120 > $B/cgroup.procs
$ sudo perf sched record -o after.data -a -- sleep 10
# launch: 1.9 s → 0.9 s;  time in D: 1.2 s → 0.25 s
```

Fix the culprit's I/O, not the victim's CPU: cap the indexer with `io.max` in its cgroup, record again and compare.



Not every "slow" is a CPU problem: tiny sch delay + long sleeps in D = storage. Throttle the culprit's I/O, not the victim's CPU. (example)

Tuning Checklist for webOS Services

Do

- **Measure first: perf sched, ftrace, Perfetto**
  measure before you change anything (p.63–64)
- **Check core types first: cpu_capacity**
  look at big / LITTLE, not just CPU numbers (p.15)
- **Use cgroups for groups of tasks**
  set policy per app and per service (p.60–61)
- **Prefer weight / uclamp before real-time**
  RT is the last resort (p.69 ①②)
- **Let EAS / misfit place bursty work on big**
  don't pin bursts yourself — leave it to the scheduler (p.56–57)
- **Keep background on LITTLE: cpuset + uclamp.max**
  "LITTLE, capped" (p.61)
- **Keep RT work short, periodic and bounded**
  only for small, periodic work like audio (p.53–54)
- **Size thread pools from allowed CPUs and capacity**
  not from nproc: the CPUs you may actually use, and their size

Don't

- **Make everything SCHED_FIFO**
  priority loses meaning; tasks starve each other and all fair tasks
- **Hard-code CPU numbers in affinity masks**
  CPU numbers and core types differ per SoC
- **Pin background work to big cores**
  it takes the big cores the UI needs (p.63 example)
- **Treat all CPUs as equal**
  count capacity, not cores (p.55)
- **Disable RT throttling in production**
  one runaway RT task can freeze the whole system
- **Busy-poll instead of blocking**
  burns CPU while waiting — steals time and power from others
- **Assume nproc == CPUs allowed in your cgroup**
  with a cpuset you may have fewer CPUs
- **Block the UI thread on an IPC reply**
  if the service is late, the UI freezes too (p.69 ④ inversion)

Rule: **fix the culprit's policy before boosting the victim.**

Putting It Together: Scheduling in a webOS System

webOS policy (Part 4)

LSM	WAM / Chromium	Media / Audio	SAM	Background
Qt event loop	web apps	GLib pipelines	app lifecycle	updates, indexing
high weight, uclamp.min	fg / bg cgroups	RT or high weight	moves apps between cgroups	low weight, LITTLE

↓

Linux scheduler (Part 3)

Classes: RT · fair · idle	Runqueues + balancing (capacity, misfit)	Wake-up placement: cache vs. wait	EAS + uclamp + schedutil	cgroups: weight, max, cpuset, uclamp

↓

Hardware (Parts 1–2)

big CPU 4–5 (L1/L2)	LITTLE CPU 0–3 (L1/L2)	shared L3 (DSU) · GPU · VDEC	one shared DRAM

Smooth webOS = **right policy** × **right scheduler mechanism** × **right placement on the hardware**.

End of Week 4

Youngjae Kim, Ph.D.

